

Formation R

Collection de packages tidyverse

Laure Velez (laure.velez@umontpellier.fr)

Mathieu Depetris (mathieu.depetris@cnrs.fr)

Monique Simier (monique.simier@ird.fr)

3 et 4 avril 2025

© RStudio



Plan de la formation

- Préambule et introduction
 - Prérequis et environnement de travail nécessaire à la formation
 - Objectifs de la formation
 - Présentation de [tidyverse](#)
- Partie 1 : Importation de données sous R avec [readr](#)
- Partie 2 : Manipulation de données avec [dplyr](#)
- Partie 3 : Nettoyage des données avec [tidyr](#)
- Partie 4 : Manipuler du texte avec [stringr](#)
- Partie 5 : Manipuler des dates avec [lubridate](#)
- Partie 6 : Visualisation graphique avec [ggplot2](#)

Formation fortement inspirée du site de [Joseph Larmarange](#)



Préambule et introduction

- Prérequis et environnement de travail nécessaire à la formation
- Objectifs de la formation
- Présentation de `tidyverse`



Prérequis et environnement de travail nécessaire à la formation

- Avoir des notions de base dans l'utilisation de R
- Avoir une instance valide de R sur son poste (durant la formation utilisation de la version 4.4.3 sous Windows)
- Pour information, nous allons utiliser les packages-version et dépendances R suivants :
 - `readr` (utilisation de la version 2.1.5),
 - `dplyr` (utilisation de la version 1.1.4),
 - `tidyr` (utilisation de la version 1.3.1),
 - `stringr` (utilisation de la version 1.5.1),
 - `lubridate` (utilisation de la version 1.9.4),
 - `ggplot2` (utilisation de la version 3.5.1),



Bonus non obligatoire

- Utilisation de l'environnement de développement **Rstudio** (utilisation de la version 2024.12.1+563 sous Windows)
- Pour les utilisateurs de Windows installation de **Rtools** (utilisation de la version 4.4)
- Installation de **Visual Studio Code** (éditeur de code source autonome).



Objectifs de la formation

- Vous présenter un panel de ce que l'on peut faire avec ces méthodes et surtout vous donner les clés pour aller plus loin
- Un expert n'est pas forcément quelqu'un qui connaît toutes les fonctions par cœur, mais qui sait écouter, comprendre et aller chercher les informations dont il a besoin
 - Pensez à jeter un coup d'œil aux [cheatsheets](#)



Présentation de Tidyverse

- Tidyverse c'est quoi ?
 - tidy + universe ou l'univers du «bien rangé» ou du rangement
 - Collection d'extensions conçues pour travailler ensemble et basées sur une philosophie commune
 - Le cœur du package inclut 9 packages
 - `ggplot2` : création de graphiques
 - `dplyr` : manipulation de données au sens large
 - `tidyr` : manipulation de données plus orientées variables
 - `readr` : import de données
 - `purrr` : fonction de vectorisation (remplacement des boucles classiques)
 - `tibble` : «évolution» du data.frame
 - `stringr` : manipulation de caractère
 - `forcats` : série d'outils pour aider dans la manipulation des facteurs
 - `lubridate` : fonctions permettant de travailler avec des dates et des heures



Présentation de Tidyverse

- Lancement de la librairie

```
1 install.packages("tidyverse")  
2 library(tidyverse)
```

- Cependant je vous déconseille de faire comme ça
 - En lançant cette commande, on charge dans R tous les packages associés à tidyverse (les 9 packages cités précédemment, mais aussi les dépendances associées)
 - Avez-vous vraiment besoin de tout cela ?
 - Potentiel impact sur vos futurs développements (surtout si vous commencez à développer des packages)
 - L'idée est d'appliquer dès maintenant les bonnes pratiques et surtout de comprendre ce que cela implique de ne pas les suivre (ce qui est aussi votre droit !)



Avant d'aller plus loin

- L'utilisation de ces packages permet de faire beaucoup de choses, mais ne rend pas tout le reste obsolète
- Il y a plusieurs stratégies pour réaliser une action, il faut juste trouver le bon compromis (connaissances, temps de calcul, contexte global de votre projet...)
- Attention aux règles et conventions de nommages
 - Règles pour nommer une variable (obligatoire) : ne pas commencer par un chiffre, pas d'espace, que des caractères alpha-numériques (A-z, 0-9) et le tiret du bas (_), ne pas utiliser de mots réservés (comme par exemple TRUE)
 - Conventions pour nommer une variable (recommandé) : uniquement des lettres minuscules, séparer les mots des _ et les noms choisis doivent pouvoir donner l'information du contenu associé
 - Une formation “bonnes pratiques en programmation” est à l'étude au sein du DEN



Partie 1 : Importation de données sous R avec readr.

[Lien cheatsheet](#)



Le package readr

- Des fonctions de base de R sont dédiées à l'import de fichiers textes de type csv (`read.csv`, `read.csv2`) ou tabulés (`read.table`) ou avec des séparateurs quelconques (`read.delim`)
- La librairie `readr` propose des fonctions similaires sur le principe mais qui diffèrent des fonctions de base sur certains aspects
- Ces fonctions sont compatibles avec l'esprit **Tidyverse** car elles permettent de générer des `tibbles`



Un tibble ... Cékoïça ???

- Le *tibble* est un *data frame* amélioré
- Ce format est géré par le package `tibble` qui fait partie du coeur de tidyverse
- Les *tibbles* contiennent un attribut *spec* qui précise les spécifications des colonnes (nom et type)
- Ils s'affichent plus intelligemment que les *data frames* : par défaut affichages uniquement des premières lignes, ainsi que quelques informations supplémentaires utiles
- Ils autorisent des noms de colonnes qui sont invalides pour les *data frames* (espaces, caractères spéciaux, nombres...)
- Ils affichent un avertissement si on essaie d'accéder à une colonne qui n'existe pas
- Ils n'ont pas de noms de lignes (*rownames*)



On peut passer du *tibble* au *dataframe* et vice-versa

- La plupart des fonctions du tidyverse acceptent des *data frames* en entrée, mais retournent un objet de classe *tibble*
- On peut convertir un *data frame* en *tibble* avec la fonction `as_tibble`, ou l'inverse avec la fonction `as.data.frame`



Les principales fonctions de la librairie readr

Les fonctions de readr les plus utilisées sont :

```
1 # Fonction générale, pour importer un fichier avec un délimiteur quelconque
2 read_delim(file, delim, ...)
3 # Import fichiers csv au format anglais (point décimal et séparateur virgule)
4 read_csv(file, ...)
5 # Import fichiers csv au format français (virgule décimale et séparateur point-virgule)
6 read_csv2(file, ...)
7 # Import fichiers au format tsv ou les valeurs sont séparées par des tabulations
8 read_tsv(file, ...)
9 # Pour lire un fichier en "chunks", c'est à dire en tranches dont on définit la taille en nombre de lignes.
10 # Cette fonction permet de lire de très gros fichiers
11 # et aussi d'appliquer une fonction (CallBack) sur les données à chaque tranche.
12 # xxx est à remplacer par csv, csv2, delim
13 read_xxx_chunked(file, callback, ...)
```



read_delim(), la fonction la plus générique

- Import de fichier avec la possibilité d'utiliser des paramètres
- Fonction générale, son fonctionnement permet une approche simple des autres fonctions plus spécifiques d'import
- Parmi toutes les possibilités, on trouve comme paramètres:
 - file : chemin d'un fichier, d'une connexion ou même des données écrites à la main,
 - delim : délimiteur utilisé pour séparer les champs
 - col_names : configuration d'une en-tête (ou pas)
 - col_types : spécification des types de colonnes
 - col_select : selection des colonnes à conserver (même langage que avec [dplyr](#))
 - na : interprétation des valeurs manquantes
 - skip : échappement de lignes lors de l'import
 - name_repair : réparation "intelligente" des noms de colonnes
- Beaucoup d'autres sont disponibles en lien avec vos besoins !



On a aussi des fonctions de raccourci

- En lien avec la nature de vos imports
- La plupart des paramètres de la fonction `read_delim` sont utilisables
 - `read_csv` et `read_csv2` pour les fichiers csv,
 - `read_tsv` pour les fichiers tsv
- On a aussi les fonctions `_chunked` pour commencer à s'amuser
 - `read_delim_chunked`, `read_csv_chunked`, `read_csv2_chunked` et `read_tsv_chunked`



La fonction inverse : write_... pour exporter un fichier

```
1 # Pour exporter en csv
2 write_csv(x, file, ...)
3 write_csv2(x, file, ...)
4 # Pour exporter en choisissant le délimiteur
5 write_delim(x, file, delim, ...)
6 # Pour exporter en tsv
7 write_tsv(x, file, ...)
```



Encore une fois des paramètres

- Notamment :
 - `x` : nom de l'objet dans l'environnement R
 - `file` : chemin du fichier que l'on veut créer (ne pas oublier l'extension)
 - `delim` : délimiteur utilisé pour séparer les champs
 - `na` : interprétation des valeurs manquantes
 - `append` : si `FALSE` écrasera un fichier existant portant le même nom que l'export, `TRUE` ajoutera nos données exportées au fichier existant (si c'est possible)
 - `col_names` : inclusion du nom des colonnes dans l'export



Autres fonctions utiles

- Extraction complète des spécifications d'un tibble avec `spec`
- `cols_condense` permet d'afficher une version simplifiée des spécifications en affichant le type par défaut le plus fréquent et liste les colonnes avec un type différent. Attention, l'entrée est un objet de type `spec`.



Import de feuilles de calcul (type excel)

- Utilisation du package `readxl`
 - Développé par l'équipe de tidyverse (extension des packages du cœur)

```
1 # Fonction générique
2 read_excel(path, ...)
3 # Fonction pour importer spécifiquement un xls (excel avant 2007) ou xlsx (extension actuelle par défaut)
4 read_xls(path, ...)
5 read_xlsx(path, ...)
```

- Globalement des arguments similaires à précédemment (`col_names`, `skip`, `col_types`, ...)
- Introduction de l'argument `sheet` et `range`
- Nécessité de bien définir les types de colonnes
- Pour les exports on utilisera `openxlsx` ou `writexl`



Import de feuilles de calcul (type google)

- Utilisation du package `googlesheets4`
 - Développé encore une fois par l'équipe de tidyverse (extension des packages du cœur)
- 3 familles de fonctions (liste non exhaustive) :
 - `read_sheet` pour lire une feuille
 - `gs4_get` ou encore `gs4_find` pour lire les métadonnées associées aux feuilles
 - `write_sheet` ou `sheet_append` pour exporter des feuilles
- On retrouve encore le même type d'arguments que précédemment



Petit point sur les types de colonne

- Très bien expliqué dans la documentation de la fonction `read_delim` au niveau de l'argument `col_types`
- Renvoi à la fonction `cols` qui permet de définir un type de colonne
 - Parait anodin, mais très important, notamment pour les processus de “verification”
 - Par exemple un chiffre sans virgule (type entier) ce n'est pas pareil qu'un chiffre à virgule (type double)
- En lien avec les fonctions d'import, on peut utiliser des raccourcis pour définir nos types de colonnes
 - Par exemple `c` pour un caractère à la place de `col_character`
 - La liste complète est disponible dans la documentation de `read_delim`
- Ici il faut garder en tête l'objectif de lever les ambiguïtés



Avantages de readr vs. les fonctions de base



readr est en général plus rapide

- Du cas par cas mais globalement mieux adapté aux “gros” fichiers (en lien avec des fonctions orientées objet)
- Par exemple (tendance sans tenir compte d’autres facteurs comme l’ordinateur) pour un fichier csv de 156Mo
 - 30.84s avec `read_csv` contre 30.00s avec `read.csv`
- Pour un fichier csv de 468Mo
 - 38.63s avec `read_csv` contre 47.56s avec `read.csv`



Le vrai avantage réside dans la polyvalence

- Beaucoup de paramétrisation possible
 - Toutes les spécifications citées précédemment
 - Un gros plus sur la sélection des colonnes via l'argument `col_select` (sélection et renommage)
 - Filtrage direct lors de l'import avec les `_chunked` ou encore l'argument `skip`
- Possibilité de lire les fichiers depuis une archive (type zip) avec `readr`
- Possibilité de lire plusieurs fichiers en une fois, même plusieurs répertoires
- Vraie dynamique d'évolution dans le futur !



Exercice 1.1

- Lire le fichier `survey_metadata` et le stocker dans un objet R `survey_metadata`,
- Lire le fichier `reef_fish_biomass` et le stocker dans un objet R `reef_fish_biomass`
- Lire le fichier `reef_fish_abundance` et le stocker dans un objet R `reef_fish_abundance`

Attention ! On ne triche pas, aucune manipulation des fichiers pré-importation



Exercice 1.1 corrigé

- Lire le fichier `survey_metadata` et le stocker dans un objet R `survey_metadata`,
- Lire le fichier `reef_fish_biomass` et le stocker dans un objet R `reef_fish_biomass`
- Lire le fichier `reef_fish_abundance` et le stocker dans un objet R `reef_fish_abundance`

Attention ! On ne triche pas, aucune manipulation des fichiers pré-importation

```
1 # solution
2 survey_metadata <- read_tsv(file = "data/survey_metadata.tsv")
3 reef_fish_biomass <- read_delim(file = "data/reef_fish_biomass.dude",
4                               delim = "|")
5 Sys.setenv("VROOM_CONNECTION_SIZE" = 500000)
6 reef_fish_abundance <- read_csv(file = "data/reef_fish_abundance.zip",
7                                col_types = list(.default = col_integer()))
```



Exercice 1.2

Lire le fichier `survey_metadata` et le stocker dans un objet R `survey_metadata` en sélectionnant le pays Indonésie, une profondeur inférieure à 20m et une visibilité supérieure à 10m.



Exercice 1.2 corrigé

Lire le fichier `survey_metadata` et le stocker dans un objet R `survey_metadata` en sélectionnant le pays Indonésie, une profondeur inférieure à 20m et une visibilité supérieure à 10m.

```
1 # solution
2 function_1_2 <- function(x, pos) subset(x, country == "Indonesia" & depth < 20 & visibility > 10)
3 survey_metadata_selection_1_2 <- read_tsv_chunked(file = "data/survey_metadata.tsv",
4                                                  callback = DataFrameCallback$new(function_1_2),
5                                                  chunk_size = 1000)
```



Partie 2 : Manipulation de données avec dplyr

[Lien cheatsheet](#)



dplyr

Pourquoi utiliser dplyr et sa syntaxe ?

- Syntaxe claire et cohérente
 - “lien” avec le SQL
 - deux “familles” de fonctions : résumé et vectorisation

summary function →

vectorized function →

- Utilisation de verbes
- Généralement plus rapides que leur équivalent sous R de base
- Permet d'enchaîner les opérations avec des “pipes”



Manipulation des lignes

Manipulation de type extraction

Type de fonctions qui retournent un sous-ensemble de lignes

- Extraction d'une ou plusieurs lignes : fonctions de la famille slice & co

```
1 # sélection d'une ou plusieurs lignes par position absolue (l'argument x est l'indice de la ou des lignes)
2 slice(.data, x, ...)
3 # sélection au hasard d'un nombre de lignes ou d'une proportion du jeu de données
4 slice_sample(.data, n/prop, ...)
5 # sélection des n dernières lignes
6 slice_tail(.data, n/prop, ...)
7 # sélection des n premières lignes
8 slice_head(.data, n/prop, ...)
9 # sélection des lignes avec les valeurs les plus petites ou grandes
10 slice_min(.data, n/prop, ...)
11 slice_max(.data, n/prop, ...)
12 # sélection des valeurs distinctes
13 distinct(.data, ...)
14 # sélection en fonction d'opérateurs logiques et/ou booléens
15 filter(.data, ...)
```



Petit focus sur les opérateurs booléens et logiques

Renvoient une valeur TRUE ou FALSE

- les communs :

```
== %in% != < > <= >= ! | & is.na()
```

- certains plus particuliers :

```
&& || xor()
```

- Attention aux priorités dans les interprétations
 - & est plus fort que |, il sera exécuté en premier
 - une solution est d'utiliser des () pour séparer vos opérateurs



Exercice 2.1

- À l'aide d'une fonction dplyr, vérifier s'il y a des données dupliquées dans le jeu de données "reef_fish_biomass"
- En plus et en une seule fois, sélectionner les données dont la variable "biomass" est différente de 0, où la famille est différente de "Mullidae" et "Apogonidae" et pour finir où la classe de taille associée est ≤ 10 . Stocker le résultat dans un objet nommé `biomass_exercice2_1_distinct_filter`.



Exercice 2.1 corrigé

- À l'aide d'une fonction dplyr, vérifier s'il y a des données dupliquées dans le jeu de données "reef_fish_biomass"
- En plus et en une seule fois, sélectionner les données dont la variable "biomass" est différente de 0, où la famille est différente de "Mullidae" et "Apogonidae" et pour finir où la classe de taille associée est ≤ 10 . Stocker le résultat dans un objet nommé `biomass_exercice2_1_distinct_filter`.

```
1 biomass_exercice2_1_distinct <- distinct(.data = reef_fish_biomass)
2 biomass_exercice2_1_distinct_filter <- filter(.data = biomass_exercice2_1_distinct,
3                                             biomass != 0
4                                             & ! family %in% c("Mullidae",
5                                             "Apogonidae")
6                                             & size_class <= 10)
```



Amélioration de la lisibilité du code

Ici on a effectué 2 opérations, mais on peut imaginer en faire beaucoup plus dans la réalité.

Grâce à la syntaxe de dplyr, on peut répondre à ce besoin de deux manières différentes.

La première est d'effectuer toutes les opérations en une fois en les emboîtant :

```
1 biomass_exercice2_1_distinct_filter <- filter(.data = distinct(.data = reef_fish_biomass),  
2                                           biomass != 0  
3                                           & ! family %in% c("Mullidae",  
4                                                         "Apogonidae")  
5                                           & size_class <= 10)
```

Cependant cela a plusieurs inconvénients. La structure est peu lisible, les opérations apparaissent dans l'ordre inverse de leur réalisation et il est difficile de voir quel paramètre se rapporte à quelle fonction.



Aller encore plus loin

Utilisation d'un nouvel opérateur baptisé pipe

- Il se note %>%
- Son fonctionnement est le suivant : si j'exécute `expr %>% f`, le résultat de l'expression `expr`, à gauche du pipe, sera passé comme premier argument à la fonction `f`, à droite du pipe, ce qui revient à exécuter `f(expr)`
- L'intérêt est que l'on peut enchaîner plusieurs pipes et surtout on simplifie la syntaxe
- Avec notre exemple précédent cela donne :

```
1 biomass_exercice2_1_distinct_filter <- distinct(.data = reef_fish_biomass) %>%  
2   filter(biomass != 0  
3         & ! family %in% c("Mullidae",  
4                           "Apogonidae")  
5         & size_class <= 10)
```



Encore quelques fonctions de manipulation de lignes

Manipulation de type “tri”

- Ordonner les lignes d'une ou plusieurs colonnes

```
1 # par défaut ordre croissant, utiliser l'argument desc pour ordre décroissant
2 arrange(.data, ...)
```

Manipulation de type “ajout”

- Ajouter une ligne dans un jeu de données

```
1 # utilisation du package tibble (dépendance)
2 # intérêt des arguments .before and .after
3 add_row(.data, ...)
```



Manipulation des variables

Extraction d'une ou plusieurs colonnes

Type de fonctions qui retournent un sous-ensemble de colonnes sous la forme d'un vecteur ou d'une table

```
1 # extraction des valeurs d'une colonne sous la forme d'un vecteur
2 # on peut utiliser des noms ou les ids
3 pull(.data, var, ...)
4 # extraction d'une colonne sous la forme d'une table
5 # bien regarder les arguments de la fonction car on peut faire beaucoup
6 select(.data, ...)
7 # Changer l'ordre ou la place d'une ou plusieurs colonnes
8 # ne pas oublier les arguments .after et .before
9 relocate(.data, ...)
```



Fonctions de vectorisation

Une des deux grandes familles de fonctions avec dplyr

- On a commencé à l'aborder de manière instinctive avec `relocate()`
- Globalement, ce sont des fonctions qui ont un vecteur en entrée et retournent un vecteur de la même taille en sortie



```
1 # créer une nouvelle variable  
2 mutate(.data, ...)
```

- Petit bonus pour renommer les variables

```
1 rename(.data, ...)
```



Exercice 2.2

- À partir du jeu de données de sortie de l'exercice 1 ([biomass_exercice2_1_distinct_filter](#))
- Créer une variable “survey_id_full” qui est la concaténation des variables survey_id, species_name et size_class
- Placer cette nouvelle colonne avant la colonne survey_id
- Supprimer la quatrième colonne du jeu de données (utiliser son indice) ainsi que la colonne reporting_name
- Renommer la variable size_class en size_class_cm
- Créer une nouvelle variable size_class_mm qui convertit la variable size_class_cm en mm
- Supprimer la variable size_class_cm
- Trier par ordre décroissant les données en fonction de la variable size_class_mm
- Utiliser des pipes à la fin pour avoir une syntaxe “propre”, stocker le resultat dans un objet nommé [biomass_exercice2_2](#) et souffler un bon coup :)



Exercice 2.2 corrigé

- À partir du jeu de données de sortie de l'exercice 1 (`biomass_exercice2_1_distinct_filter`)
- Créer une variable “survey_id_full” qui est la concaténation des variables `survey_id`, `species_name` et `size_class`
- Placer cette nouvelle colonne avant la colonne `survey_id`
- Supprimer la quatrième colonne du jeu de données (utiliser son indice) ainsi que la colonne `reporting_name`
- Renommer la variable `size_class` en `size_class_cm`
- Créer une nouvelle variable `size_class_mm` qui convertit la variable `size_class_cm` en mm
- Supprimer la variable `size_class_cm`
- Trier par ordre décroissant les données en fonction de la variable `size_class_mm`
- Utiliser des pipes à la fin pour avoir une syntaxe “propre”, stocker le resultat dans un objet nommé `biomass_exercice2_2` et souffler un bon coup :)

```
1 # solution
2 biomass_exercice2_2 <- mutate(.data = biomass_exercice2_1_distinct_filter,
3                               survey_id_full = paste(survey_id,
4                                                       species_name,
5                                                       size_class,
6                                                       sep = "_")) %>%
7   relocate(survey_id_full,
8             .before = survey_id) %>%
9   select(-4,
10         -reporting_name) %>%
11   rename(size_class_cm = size_class) %>%
12   mutate(size_class_mm = size_class_cm * 10) %>%
13   select(-size_class_cm) %>%
14   arrange(desc(size_class_mm))
```



Fonctions de résumé

La deuxième grande famille

- Fonctions qui vont créer une nouvelle table avec des résumés statistiques. De manière globale, les fonctions associées vont utiliser en entrée des vecteurs et en sortie une ou plusieurs valeurs (en nombre inférieur au nombre de vecteurs d'entrée)



- Fonction clé

```
1 summarise(.data, ...)  
2 # Beaucoup, beaucoup d'options et de possibilités  
3 # Compter le nombre de valeurs/lignes  
4 n()  
5 # Compter le nombre de valeurs/lignes distinctes  
6 n_distinct()  
7 # Faire une moyenne  
8 mean()  
9 # Faire une somme, option na.rm disponible  
10 sum()  
11 # Utiliser des notions de position  
12 first()  
13 last()  
14 # Ou de rang  
15 quantile()  
16 min()  
17 max()
```



Introduction à la notion de groupe

- Pour le moment on a fait des opérations sur l'ensemble du jeu de données, pas très pertinent non ?
- Fonction de création de groupe

```
1 # on peut définir nos groupes en fonction des modalités des variables
2 # on peut aussi utiliser des arguments des fonctions quand c'est proposé, exemple avec .by de summarise()
3 group_by(.data, ...)
4 # ou alors en isolant chaque ligne (utile surtout pour les fonctions qui ne vectorise pas)
5 rowwise()
```

- Attention à ce que vous faites ! Par défaut, les groupes sont gardés en mémoire dans votre jeu de données de sortie (lisez les messages de R). Pour y remédier :

```
1 # via un summarise (argument experimental, attention à la pérennité)
2 .groups
3 # de manière plus générale
4 ungroups()
```

- Cette notion de groupe peut être utilisée avec d'autres fonctions (comme un mutate)



Exercice 2.3

- À partir du jeu de données de sortie de l'exercice 2.2 ([biomass_exercice2_2](#))
- Effectuer la somme de la biomasse (variable `biomass`) en fonction de la variable `survey_id_full` créée précédemment
- Pour `survey_id_full` répétés, ne garder que la première ligne de chaque répétition. Attention à bien conserver toutes les colonnes associées (une fois la fonction “trouvée” regarder du côté des arguments en option)
- Créer un nouveau jeu de données qui exprime le nombre d'échantillons disponibles par famille et vérifier la cohérence du nombre total obtenu avec le jeu de données de départ
- Stocker tout ça dans un objet nommé [biomass_exercice2_3](#)



Exercice 2.3 corrigé

- À partir du jeu de données de sortie de l'exercice 2.2 (`biomass_exercice2_2`)
- Effectuer la somme de la biomasse (variable `biomass`) en fonction de la variable `survey_id_full` créée précédemment
- Pour `survey_id_full` répétés, ne garder que la première ligne de chaque répétition. Attention à bien conserver toutes les colonnes associées (une fois la fonction “trouvée” regarder du côté des arguments en option)
- Créer un nouveau jeu de données qui exprime le nombre d'échantillons disponibles par famille et vérifier la cohérence du nombre total obtenu avec le jeu de données de départ
- Stocker tout ça dans un objet nommé `biomass_exercice2_3`

```
1 # solution
2 biomass_exercice2_3 <- biomass_exercice2_2 %>%
3   dplyr::group_by(survey_id_full) %>%
4   mutate(sum_biomass = sum(x = biomass)) %>%
5   ungroup() %>%
6   distinct(survey_id_full,
7            .keep_all = TRUE)
8 biomass_exercice2_3_sample_verification <- dplyr::group_by(.data = biomass_exercice2_2,
9                                                            family) %>%
10  summarise(nb_sample_family = n()) %>%
11  mutate(total = sum(nb_sample_family))
```



Fonction bonus très utile

case_when : généralisation vectorisée de if(s)

- Vectorisation de multiples if_else sous R et CASE WHEN en SQL
- Améliore grandement la lisibilité de vos fonctions
- Syntaxe de base :

```
1 case_when(  
2   opérateur(s) booléen(s) ou logique(s) ~ output1,  
3   opérateur(s) booléen(s) ou logique(s) ~ output2,  
4   ....  
5   opérateur(s) booléen(s) ou logique(s) ~ output_n,  
6   TRUE ~ output si rien ne marche au dessus  
7 )
```

- Pensez bien à toujours mettre un TRUE (=valeur par défaut)
- Attention à ne pas mélanger les types, il existe différents NA :
 - NA_integer_
 - NA_real_
 - NA_complex_
 - NA_character_



Combinaison de jeux de données 1

Jointures “simples”

- Il doit y avoir une cohérence entre le nombre de colonnes ou de lignes
- Attention, aucune vérification de cohérence des jointures

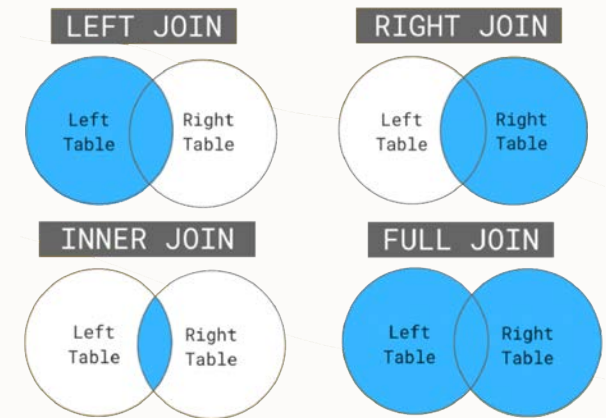
```
1 # "Coller" deux jeux de données
2 bind_cols()
3 bind_rows()
```



Combinaison de jeux de données 2

Jointures de type relationnel

- Très proche du SQL
- Très puissantes et permettent de se poser les bonnes questions !
- Attention à bien spécifier l'argument `by` (ne pas laisser place à l'interprétation !)
 - On peut spécifier une seule ou plusieurs colonnes si les noms sont identiques entre les deux jeux de données
 - Sinon on spécifie le nom du jeu de données à “gauche” et celui qui est à “droite”



```
1 # jointure à gauche
2 left_join(x, y, by, ...)
3 # jointure à droite
4 right_join(x, y, by, ...)
5 # jointure commune
6 inner_join(x, y, by, ...)
7 # jointure complete
8 full_join(x, y, by, ...)
```



Exercice 2.4

- À partir du jeu de données de sortie de l'exercice 2.3 ([biomass_exercice2_3](#))
- Faire la “meilleure” jointure avec le jeu de données [survey_metadata](#) et expliquer votre choix
- Pour vous aider :
 - Y-a-t-il un problème avec le jeu de données [survey_metadata](#) ?
 - Réfléchissez en termes du nombre de lignes obtenues pour chaque fonction de jointure
- Stocker le résultat dans un objet nommé [biomass_survey_exercice2_4](#)



Exercice 2.4 corrigé

- À partir du jeu de données de sortie de l'exercice 2.3 (`biomass_exercice2_3`)
- Faire la “meilleure” jointure avec le jeu de données `survey_metadata` et expliquer votre choix
- Pour vous aider :
 - Y-a-t-il un problème avec le jeu de données `survey_metadata` ?
 - Réfléchissez en termes du nombre de lignes obtenues pour chaque fonction de jointure
- Stocker le résultat dans un objet nommé `biomass_survey_exercice2_4`

```
1 # solution
2 survey_metadata_distinct <- distinct(.data = survey_metadata)
3 biomass_survey_exercice2_4 <- biomass_exercice2_3 %>%
4   left_join(survey_metadata_distinct,
5             by = "survey_id")
```



Partie 3 : Nettoyage des données avec tidyr

[Lien cheatsheet](#)



Tidyr

L'objectif du package tidyr est de fournir des fonctions pour arranger et nettoyer ses données dans le but de les convertir dans un format “tidy”.

Les différents verbes viennent s'ajouter à tous ceux que nous avons déjà vus avec `readr` et `dplyr` et ils s'intègrent parfaitement dans la série de pipe `%>%` permettant d'enchaîner les opérations.

- Les données “tidy”, qu'est ce que c'est?
 - 1 variable = 1 colonne
 - 1 observation = 1 ligne/case
 - 1 type d'observation par table



Les principales fonctions de tidyr

Modelage des tableaux

`pivot_wider()` et `pivot_longer()` sont les successeurs des fonctions en base R du package `reshape` puis `reshape2`. Et se nommaient `spread()` et `gather()` dans les premières versions de `tidyr`

```
1 # Transformer un tableau long en format large
2 pivot_wider(names_from, values_from)
3 # Transformer un tableau large en format long
4 pivot_longer(cols, names_to, values_to, ...)
```



Exercice 3.1

- Observez les données de biomasse ([biomass_survey_exercice2_4](#)) et d'abondance. Les données sont elles tidy ? Si non, pourquoi ?



Exercice 3.1 corrigé

- Observez les données de biomasse ([biomass_survey_exercice2_4](#)) et d'abondance. Les données sont-elles tidy ? Si non, pourquoi ?

```
1 # solution
2 head(x = biomass_survey_exercice2_4)
3 View(x = biomass_survey_exercice2_4)
```

Non les données d'abondance ne sont pas tidy, car elles sont en format "wide"



Exercice 3.2

- Transformez les données d'abondance en format tidy. Les nouvelles colonnes créées seront `sp_size` et `abundance`
- Pensez à bien regarder tous les arguments des fonctions
- Stocker le résultat dans un objet nommé `abundance_exercice3_2`



Exercice 3.2 corrigé

- Transformez les données d'abondance en format tidy. Les nouvelles colonnes créées seront `sp_size` et `abundance`
- Pensez à bien regarder tous les arguments des fonctions
- Stocker le résultat dans un objet nommé `abundance_exercice3_2`

```
1 # solution
2 abundance_exercice3_2 <- reef_fish_abundance %>%
3   pivot_longer(.,
4     cols = 2:ncol(.),
5     names_to = "sp_size",
6     values_to = "abundance")
```



Gestion des valeurs manquantes “NA”

```
1 # Remplacer des "NA" par des valeurs spécifiques
2 replace_na(data, replace, ...)
3 # Supprimer les lignes contenant des "NA"
4 drop_na(data, ...)
5 # Remplir les "NA" en utilisant les valeurs précédentes ou suivantes de la même colonne
6 fill(data, ..., .direction)
```



Exercice 3.3

- A partir du jeu de données issu de l'exercice 3.2 (`abundance_exercice3_2`) supprimez les lignes ayant des valeurs d'abondance égales à NA
- Stocker le résultat dans un objet nommé `abundance_exercice3_3`



Exercice 3.3 corrigé

- A partir du jeu de données issu de l'exercice 3.2 (`abundance_exercice3_2`) supprimez les lignes ayant des valeurs d'abondance égales à NA
- Stocker le résultat dans un objet nommé `abundance_exercice3_3`

```
1 # solution
2 abundance_exercice3_3 <- abundance_exercice3_2 %>%
3   drop_na(.,
4     abundance)
```



Combinaison des variables

```
1 # Séparer une colonne en plusieurs colonnes
2 separate(data, col, into)
3 # Créer une seule colonne à partir de plusieurs colonnes
4 unite(col, ..., sep)
5 # Générer toutes les combinaisons 2 à 2 possibles entre les variables définies
6 expand(data, ...)
```



Exercice 3.4

- Vérifiez s'il y a des données dupliquées dans le jeu de données issu de l'exercice 3.3 ([abundance_exercice3_3](#))
- Créez une nouvelle variable “survey_id_full” qui est l'union entre le survey_id et sp_size avec comme séparateur un “_”.
- En vous servant des fonctions vues depuis le début de la formation créez un seul tableau regroupant biomasse et abondance . Pensez à enchaîner les opérations
- Stocker le résultat dans un objet nommé [abundance_biomass_survey_exercice3_4](#)



Exercice 3.4 corrigé

- Vérifiez s'il y a des données dupliquées dans le jeu de données issu de l'exercice 3.3 (`abundance_exercice3_3`)
- Créez une nouvelle variable "survey_id_full" qui est l'union entre le survey_id et sp_size avec comme séparateur un "_".
- En vous servant des fonctions vues depuis le début de la formation créez un seul tableau regroupant biomasse et abondance . Pensez à enchaîner les opérations
- Stocker le résultat dans un objet nommé `abundance_biomass_survey_exercice3_4`

```
1 # solution
2 abundance_biomass_survey_exercice3_4 <- abundance_exercice3_3 %>%
3   distinct() %>%
4   unite(.,
5         col = "survey_id_full",
6         c(survey_id,
7           sp_size),
8         sep = "_") %>%
9   right_join(.,
10             biomass_survey_exercice2_4,
11             by = "survey_id_full") %>%
12   relocate(abundance,
13            .after = biomass)
```



Exercice 3.5

- A partir du jeu de données précédent (`abundance_biomass_survey_exercice3_4`)
- Créez 2 nouvelles colonnes `genus` et `species` à partir de `species_name`
- Supprimez les colonnes de `has_pq_scores_in_db` jusqu'à `methods`
- `has_pq_scores_in_db` est en position 23
- Stocker le résultat dans un objet nommé `abundance_biomass_survey_exercice3_5`



Exercice 3.5 corrigé

- A partir du jeu de données précédent (`abundance_biomass_survey_exercice3_4`)
- Créez 2 nouvelles colonnes `genus` et `species` à partir de `species_name`
- Supprimez les colonnes de `has_pq_scores_in_db` jusqu'à `methods`
- `has_pq_scores_in_db` est en position 23
- Stocker le résultat dans un objet nommé `abundance_biomass_survey_exercice3_5`

```
1 # solution
2 abundance_biomass_survey_exercice3_5 <- abundance_biomass_survey_exercice3_4 %>%
3   separate(.,
4             species_name,
5             into = c("genus",
6                     "species")) %>%
7   select(-c(23:ncol(.)))
```



Partie 4 : Manipuler du texte avec stringr

[Lien cheatsheet](#)



Conçu pour manipuler les chaînes de caractères “facilement”

- En plus d’être plus faciles à utiliser, les noms des fonctions représentent bien ce qu’elles permettent de faire
 - elles commencent toutes par `str_`
 - les images de la cheatsheet aident beaucoup pour choisir les fonctions en rapport avec notre besoin
- Ouvre le monde des expressions régulières
- Peut être combiné avec du langage tidyverse (par exemple avec `dplyr`)
- Pour faire simple on a 7 grandes familles de fonctions
 - les fonctions de détection
 - les fonctions de sélection ou d’extraction
 - les fonctions de gestion des tailles de caractères
 - les fonctions de manipulation dites `mutate` (ça vous rappelle quelque chose ?)
 - les fonctions de jointures et de division
 - les fonctions d’ordre
 - les fonctions supports



Beaucoup d'arguments communs

- De manière générale
 - `string` : le vecteur d'entrée
 - `pattern` : le motif que l'on va chercher à regarder
 - `replacement` : la valeur de remplacement
- De manière plus spécifique
 - `start` et `end` : emplacement ou indice de position dans une chaîne de caractères
 - `negate` : si TRUE, le résultat sera l'inverse de ce que l'on demande
 - `sep` : caractère(s) à insérer dans le vecteur d'entrée
 - `collapse` : souvent confondu avec l'argument `sep`. Caractère(s) qui vont combiner les résultats (= vecteur d'entrée avec plusieurs éléments) en une seule et unique chaîne de caractères.
 - `decreasing` : utilisé pour les fonctions d'ordre ou de tri



Fonctions de détection

Renvoient une réponse (pas d'extraction)

```
1 # détection d'un motif dans une chaine de caractères
2 str_detect(string, pattern, negate) et str_like(string, pattern, ...)
3
4 # détection d'un motif au début ou à la fin d'une chaine de caractères
5 str_starts(string, pattern, negate) et str_ends(string, pattern, negate)
6
7 # détection des lignes contenant le motif
8 str_which(string, pattern, negate)
9
10 # détection de la position du motif dans la chaine de caractères (début et fin)
11 # sans le _all la détection passe à la chaine suivante dès qu'un motif est trouvé
12 str_locate(string, pattern) et str_locate_all(string, pattern)
13
14 # comptage du nombre de fois que le motif est détecté dans une chaine de caractères
15 str_count(string, pattern)
```



Fonctions de sélection ou d'extraction

Renvoient une chaîne de caractères

```
1 # extraction d'une chaîne à un ou des endroits bien précis
2 # utilisation d'indices de position
3 str_sub(string, start, end) et str_sub_all(string, start, end)
4
5 # extraction des chaînes, complètes, contenant un motif défini
6 str_subset(string, pattern, negate)
7
8 # extraction d'un motif dans une chaîne de caractères et retour sous forme de vecteur
9 # par défaut la détection passe à la chaîne suivante dès qu'un motif est trouvé
10 str_extract(string, pattern, ...) et str_extract_all(string, pattern, ...)
11
12 # extraction d'un motif dans une chaîne de caractères et retour sous forme de matrix
13 # sans le _all la détection passe à la chaîne suivante dès qu'un motif est trouvé
14 str_match(string, pattern) et str_match_all(string, pattern)
```



Exercice 4.1

- À partir du jeu de données `biomass_survey_exercice2_4`, extraire les numéros de lignes correspondant aux données des échantillons de la famille `Embiotocidae` puis utiliser ces indices pour sélectionner ces lignes et les variables associées (utiliser la fonction `slice` de `dplyr`). Stocker le résultat dans un objet `biomass_survey_exercice4_1`.
- compter combien nous avons de données d'occurrence d'espèces du genre `Embiotoca` dans ce nouveau jeu de données
- en utilisant une négation, extraire toutes les localisations qui sont différentes de `British Columbia`



Exercice 4.1 corrigé

- À partir du jeu de données `biomass_survey_exercice2_4`, extraire les numéros de lignes correspondant aux données des échantillons de la famille `Embiotocidae` puis utiliser ces indices pour sélectionner ces lignes et les variables associées (utiliser la fonction `slice` de `dplyr`). Stocker le résultat dans un objet `biomass_survey_exercice4_1`.
- compter combien nous avons de données d'occurrence d'espèces du genre `Embiotoca` dans ce nouveau jeu de données
- en utilisant une négation, extraire toutes les localisations qui sont différentes de `British Columbia`

```
1 # solution
2 biomass_survey_exercice4_1 <- slice(.data = biomass_survey_exercice2_4,
3                                   str_which(string = biomass_survey_exercice2_4$family,
4                                             pattern = "Embiotocidae"))
5 sum_data_embiotoca_exercice4_1 <- sum(str_count(string = biomass_survey_exercice4_1$species_name,
6                                             pattern = "Embiotoca."))
7 location_biomass_survey_exercice4_1 <- unique(str_subset(string = biomass_survey_exercice4_1$location,
8                                                           pattern = "British Columbia",
9                                                           negate = TRUE))
```



Fonctions de gestion des tailles de caractères

```
1 # largeur/nombre de caractères
2 # attention aux espaces et caractères masqués
3 str_length(string)
4
5 # ajout "d'espace" à une chaîne de caractères
6 str_pad(string, width, side, ...)
7
8 # tronquer une chaîne de caractères à un nombre fixe de caractères
9 str_trunc(string, width, side, ...)
10
11 # supprimer des espaces
12 # au début et/ou à la fin d'une chaîne de caractères
13 str_trim(string, side)
14 # au début, à la fin et réduction des espaces à un seul
15 str_squish(string)
```



Fonctions de manipulation

En lien avec des manipulations de type vectorisation (= conservation de la structure des données d'entrée)

```
1 # remplacement du motif via un assignation
2 str_sub(string, start, end) <- value
3
4 # remplacement d'un motif par un autre
5 # sans le _all le remplacement passe à la chaîne suivante dès qu'un motif est trouvé
6 str_replace(string, pattern, replacement) et str_replace_all(string, pattern, replacement)
7
8 # suppression d'un motif
9 # sans le _all la suppression passe à la chaîne suivante dès qu'un motif est trouvé
10 str_remove(string, pattern) et str_remove_all(string, pattern)
11
12 # manipulation de taille de caractère (majuscule, minuscule, ...)
13 # attention à la spécification de la langue et au nom, un titre n'est pas forcément un titre que vous cherchez
14 str_to_lower(string, ...)
15 str_to_upper(string, ...)
16 str_to_title(string, ...)
17 str_to_sentence(string, ...)
```



Exercice 4.2

- À partir du jeu de données `biomass_survey_exercice4_1`, créer une nouvelle variable, `site_name_length_ori`, contenant la taille des chaînes de caractères de la variable `site_name`
 - Nettoyer la variable `site_name` en enlevant les espaces en trop (avant, après et au milieu) et la stocker dans une variable `site_name_clean`
 - Enlever les majuscules de notre variable `site_name_clean` sauf celles en début de chaîne
 - Stocker le résultat dans un objet `biomass_survey_exercice4_2`
-



Exercice 4.2 corrigé

- À partir du jeu de données `biomass_survey_exercice4_1`, créer une nouvelle variable, `site_name_length_ori`, contenant la taille des chaînes de caractères de la variable `site_name`
- Nettoyer la variable `site_name` en enlevant les espaces en trop (avant, après et au milieu) et la stocker dans une variable `site_name_clean`
- Enlever les majuscules de notre variable `site_name_clean` sauf celles en début de chaîne
- Stocker le résultat dans un objet `biomass_survey_exercice4_2`

```
1 # solution
2 biomass_survey_exercice4_2 <- biomass_survey_exercice4_1 %>%
3   mutate(site_name_length_ori = str_length(site_name),
4           site_name_clean = str_squish(site_name),
5           site_name_clean = str_to_sentence(site_name_clean)) %>%
6   relocate(site_name_length_ori,
7             site_name_clean,
8             .after = site_name)
```



Fonctions de jointures et de division

Très pratique surtout si utilisation via langage tidyverse

```
1 # jointure de plusieurs chaines de caractères en une seule (conservation structure en lignes)
2 str_c()
3 # jointure de plusieurs chaines de caractères en un seul vecteur (utiliser argument collapse)
4 str_flatten(string, collapse)
5 # répétition d'une chaîne de caractères
6 str_dup(string, times)
7 # suppression des chaînes de caractères répétées
8 str_unique(string, ...)
9 # division d'une chaîne de caractères en fonction d'un motif, le résultat est une liste de chaînes
10 str_split(string, pattern, ...)
11 # argument n pour spécifier le nombre possible de correspondances
12 # si on veut retourner un vecteur (ne fonctionne que sur une chaîne unique)
13 str_split_1(string, pattern)
14 # si on veut retourner une matrice
15 str_split_fixed(string, pattern, n)
16 # si on veut extraire uniquement une partie de la chaîne
17 str_split_i(string, pattern, i)
18 # si on veut faire des interpolations dans des chaînes de caractères (similaire aux fonctions type paste())
19 # utilisation de {} dans notre expression
20 str_glue()
21 # variante si on veut l'utiliser dans le cadre d'une pipe
22 str_glue_data()
```



Fonctions d'ordre

Très simple, mais attention à la définition des arguments, notamment le `locale` qui va définir l'environnement de l'utilisateur.

```
1 # retourne un vecteur des indices ordonnés
2 str_order(x)
3
4 # trier un vecteur
5 str_sort(x)
```



Fonctions supports

A voir comme un petite boîte à outils

```
1 # pour jouer avec l'encodage
2 str_conv(string, encoding)
3
4 # pour jouer avec la visualisation de certains caractères (notamment sous la console de R)
5 str_view(string, ...)
6
7 # pour déterminer si deux chaines de caractères sont égales
8 str_equal(x, y, ...)
9
10 # formate une chaine de caractères en paragraphes
11 str_wrap(string, ...)
```



Exercice 4.3

- À partir du jeu de données `biomass_survey_exercice4_2`, créer deux nouvelles variables, `species_name_first` et `species_name_second`, qui contiennent une division de la variable `species_name` en deux mots (on utilisera la valeur `[.]` pour spécifier `.` comme séparateur, bizarre non ?)
- Créer une variable `biomass_percentage` qui contiendra la biomasse exprimée en % (avec % à la fin)
- Fusionner dans une nouvelle variable `description` les variables `species_name_first`, `species_name_second` et `biomass_percentage` pour obtenir une phrase correcte (avec les bons espaces aux bons endroits, la variable `biomass_percentage` en parenthèses et suivi de la chaîne de caractère `of total biomass`)
- Ayant une très mauvaise vision et un écran très petit, je souhaiterais que l'affichage de la variable `description` ne dépasse pas 20 caractères par ligne. Trouver un moyen de faire ça et stocker le résultat dans une nouvelle variable `description_shinny`. Attention, il faut parfois regarder au-delà de ce que l'on voit dans la console de R...
- Stocker le résultat dans un objet `biomass_survey_exercice4_3`



Exercice 4.3 corrigé

- À partir du jeu de données `biomass_survey_exercice4_2`, créer deux nouvelles variables, `species_name_first` et `species_name_second`, qui contiennent une division de la variable `species_name` en deux mots (on utilisera la valeur `[.]` pour spécifier `.` comme séparateur, bizarre non ?)
- Créer une variable `biomass_percentage` qui contiendra la biomasse exprimée en % (avec % à la fin)
- Fusionner dans une nouvelle variable `description` les variables `species_name_first`, `species_name_second` et `biomass_percentage` pour obtenir une phrase correcte (avec les bons espaces aux bons endroits, la variable `biomass_percentage` en parenthèses et suivi de la chaîne de caractère `of total biomass`)
- Ayant une très mauvaise vision et un écran très petit, je souhaiterais que l'affichage de la variable `description` ne dépasse pas 20 caractères par ligne. Trouver un moyen de faire ça et stocker le résultat dans une nouvelle variable `description_shinny`. Attention, il faut parfois regarder au-delà de ce que l'on voit dans la console de R...
- Stocker le résultat dans un objet `biomass_survey_exercice4_3`

```
1 # solution
2 biomass_survey_exercice4_3 <- biomass_survey_exercice4_2 %>%
3   mutate(species_name_first = str_split_i(string = species_name,
4     pattern = "[.]",
5     i = 1),
6     species_name_second = str_split_i(string = species_name,
7     pattern = "[.]",
8     i = 2),
9     biomass_percentage = str_glue("{round(x = biomass * 100 / sum_biomass)}%"),
10    description = str_c(species_name_first,
11      species_name_second,
12      str_c("(",
13        biomass_percentage,
14        " of total biomass)"),
15      sep = " "),
16    description_shinny = str_wrap(string = description,
17      width = 20))
```



Les expressions régulières

Outil très très puissant !

- Il faut savoir que cela existe, mais demande un peu de patience et d'entraînement pour être utilisé
- Regardez le [deuxième volet](#) du cheatsheet de stringr
- Dans l'exercice 4.3, on a utilisé `[.]` pour un motif, c'est une expression régulière
- Permet de “sélectionner” de manière “dynamique” des patterns dans vos données
- Le plus difficile est de formuler de la meilleure façon ce que vous voulez
 - Il y a souvent plusieurs façons d'arriver à ce que vous voulez
- Parfois le plus simple est de reformuler ce que vous voulez faire, ou de le faire de manière séquentielle
- Utiliser une IA peut vous faciliter la tâche, mais attention aux “boucles infernales” et surtout prenez du recul lors de vos échanges



Exercice bonus 4.4

- Dans le jeu données précédent, `biomass_survey_exercice4_3`, je veux séparer les chiffres et les lettres de la variable `site_code`. Comment trouver un motif qui marche aussi bien pour la valeur `USWC1` et `BC3` ?
 - Avec l'utilisation des expressions régulières je vais pouvoir lui dire “je veux que tu cherche un motif qui commence par lettre, une ou plusieurs, mais qui s'arrête à un chiffre” (même raisonnement avec les chiffres)
 - Stocker le résultat dans un objet `biomass_survey_exercice4_4`
-



Exercice bonus 4.4 corrigé

- Dans le jeu données précédent, `biomass_survey_exercice4_3`, je veux séparer les chiffres et les lettres de la variable `site_code`. Comment trouver un motif qui marche aussi bien pour la valeur `USWC1` et `BC3` ?
- Avec l'utilisation des expressions régulières je vais pouvoir lui dire "je veux que tu cherche un motif qui commence par lettre, une ou plusieurs, mais qui s'arrête à un chiffre" (même raisonnement avec les chiffres)
- Stocker le résultat dans un objet `biomass_survey_exercice4_4`

```
1 biomass_survey_exercice4_4 <- biomass_survey_exercice4_3 %>%  
2   mutate(site_code_letters = str_extract(string = site_code,  
3                                           pattern = "^[[:upper:]]+"),  
4           site_code_numbers = str_extract(string = site_code,  
5                                           pattern = "[[:digit:]]+$")) %>%  
6   relocate(site_code_letters,  
7             site_code_numbers,  
8             .after = site_code)
```



Partie 5 : Manipuler des dates avec lubridate

[Lien cheatsheet](#)



Rend la manipulation des dates et du temps presque agréable sous R

- Analyse semi-automatique des dates et des heures
- Les fonctions vont essayer de trouver le séparateur et le format
- Possibilité de rajouter l'indicateur de fuseau horaire via l'argument tz

```
1 library(lubridate)
2 # Identifie l'ordre de l'année (y), le mois (m), le jour (d), l'heure (h), la minute (m) et la seconde (s) d'un
3 ymd()
4 mdy()
5 dmy()
6 ymd_hms()
7 # Beaucoup de possibilités, jeter un coup d'oeil sur le cheatsheet
```



Manipulation des éléments date-temps

Modification et extraction

```
1 test_date_time <- ymd_hms("2022-12-05 12:00:00", tz = "CET")
2 # pour une extraction des informations
3 second(x)
4 hour(x)
5 day(x)
6 wday(x)
7 month(x)
8 ...
9 # pour modifier les informations, utiliser les fonctions précédentes avec une assignation (<-)
10 # par exemple
11 hour(x = test_date_time) <- 6
```



Tour d'horizon

- Les fuseaux horaires, un cauchemar par le passé, maintenant ça va mieux
- Deux fonctions clés

```
1 # Pour exprimer un élément dans un autre fuseau horaire
2 with_tz(time, tzone, ...)
3 # Pour forcer le changement d'un fuseau horaire mais en conservant la même heure
4 force_tz(time, tzone, ...)
```

- 3 classes pour calculer des intervalles de temps
- Periode, durée et intervalle
- Le plus “dur” est de définir votre date-temps, après c’est une syntaxe mathématique



Bonus

- Les fonctions sont vectorisables !
- Sur certains jeux de données, les dates-temps sont stockées en nombre de secondes depuis 1970-01-01 00:00:00 UTC. Utiliser cette fonction dans ce cas-là :

```
1 as_datetime(x, ...)
```

- Possibilité de faire des arrondis

```
1 # arrondi inférieur sur "l'unité" la plus proche
2 floor_date(x, ...)
3
4 # arrondi inférieur sur "l'unité" la plus proche
5 round_date(x, ...)
6
7 # arrondi supérieur sur "l'unité" la plus proche
8 ceiling_date(x, ...)
```

- De définir des objets pour les manipuler

```
1 # en lien avec les paramètres du système
2 now()
3 today()
4
5 # en lien avec des paramètres
6 dyears(x)
7 dmonths(x)
8 dweeks(x)
9 ...
```



Exercice 5.1

- À partir du jeu de données `biomass_survey_exercice4_3`, créer une variable `survey_date_as_date` qui est la conversion de la variable `survey_date` en date
- Créer la variable `survey_year` qui contient l'année de la variable `survey_date_as_date`
- Calculer la différence de temps entre aujourd'hui et la variable `survey_date_as_date`. Stocker l'information dans une nouvelle colonne `duration_since_now`
- Créer une variable `next_survey_date` qui ajoute 12 mois à la variable `survey_date_as_date`
- Regarder plus en détail la variable `next_survey_date`, qu'a-t-elle de particulier ?
- Question bonus en lien avec la question précédente, définir l'heure des valeurs de la variable `survey_date_as_date` à 12h et la stocker dans une variable `next_survey_date_correct`
- Stocker le résultat dans une variable `biomass_survey_exercice5_1`



Exercice 5.1 corrigé

- À partir du jeu de données `biomass_survey_exercice4_3`, créer une variable `survey_date_as_date` qui est la conversion de la variable `survey_date` en date
- Créer la variable `survey_year` qui contient l'année de la variable `survey_date_as_date`
- Calculer la différence de temps entre aujourd'hui et la variable `survey_date_as_date`. Stocker l'information dans une nouvelle colonne `duration_since_now`
- Créer une variable `next_survey_date` qui ajoute 12 mois à la variable `survey_date_as_date`
- Regarder plus en détail la variable `next_survey_date`, qu'a-t-elle de particulier ?
- Question bonus en lien avec la question précédente, définir l'heure des valeurs de la variable `survey_date_as_date` à 12h et la stocker dans une variable `next_survey_date_correct`
- Stocker le résultat dans une variable `biomass_survey_exercice5_1`

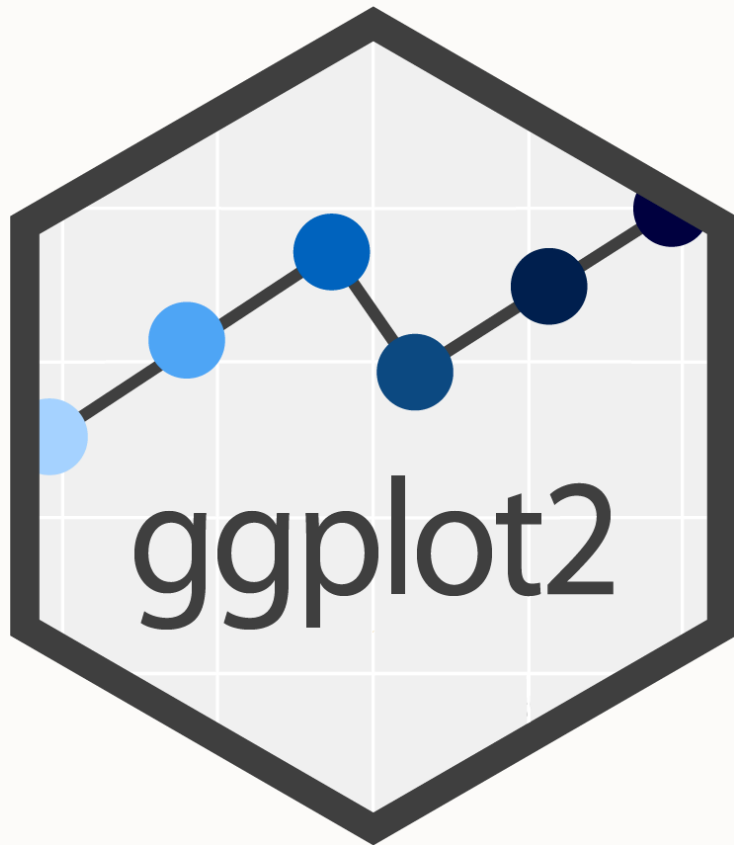
```
1 # solution
2 biomass_survey_exercice5_1 <- biomass_survey_exercice4_3 %>%
3   mutate(survey_date_as_date = dmy(survey_date),
4          survey_year = year(x = survey_date_as_date),
5          duration_since_now = today() - survey_date_as_date,
6          next_survey_date = survey_date_as_date + dmonths(x = 12),
7          next_survey_date_correct = next_survey_date %>%
8            {hour(.) <- 12; .}) %>%
9   relocate(survey_date_as_date,
10            next_survey_date,
11            survey_year,
12            duration_since_now,
13            next_survey_date_correct,
14            .after = survey_date)
```

Explication : `{ ... }` définit un bloc qui sera exécuté dans l'ordre, `;` permet de séparer les actions, `hour(.) <- 12` modifie l'heure de l'objet `.` en 12 et `.` retourne l'objet modifié



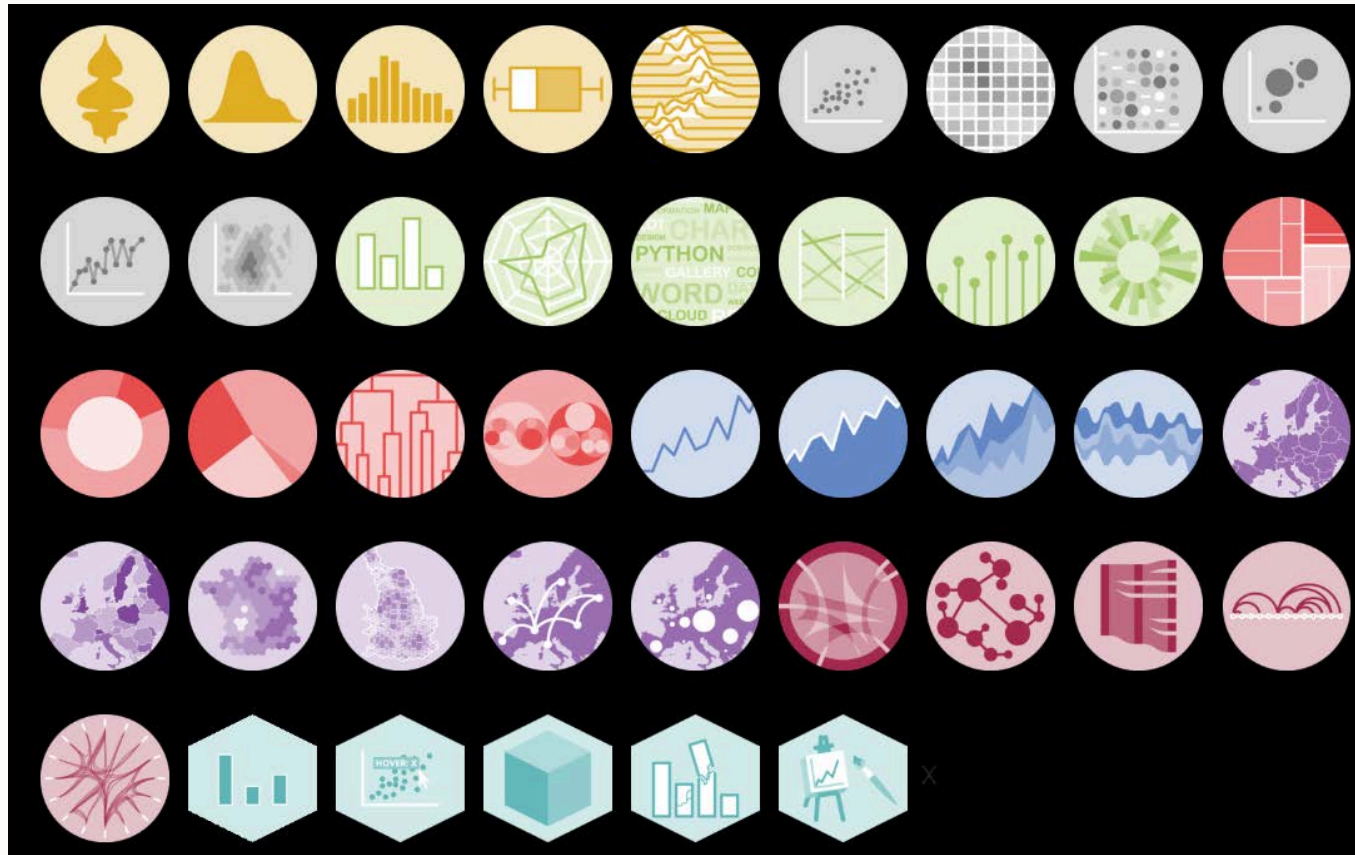
Partie 6 : Visualisation des données avec ggplot2

[Lien cheatsheet](#)



Petit aperçu des possibilités :

Le site **From Data To Viz**, une pépite

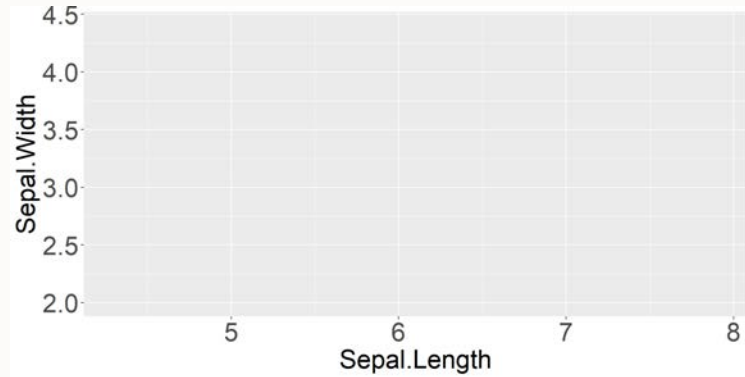


Ggplot c'est comme la chirurgie *aesthétique* : une succession de couches

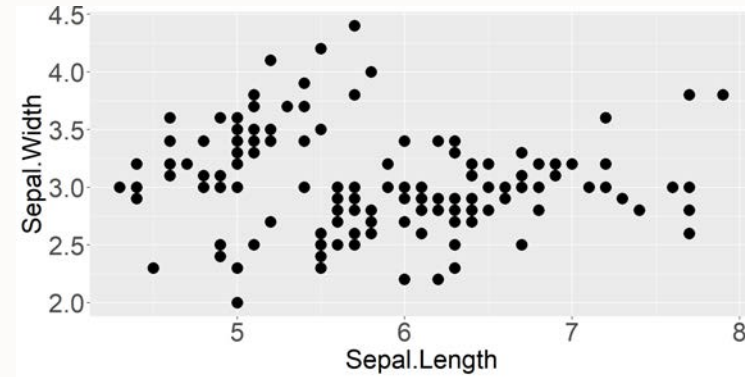
Au sein de ggplot pour enchaîner les opérations on utilise l'opérateur +



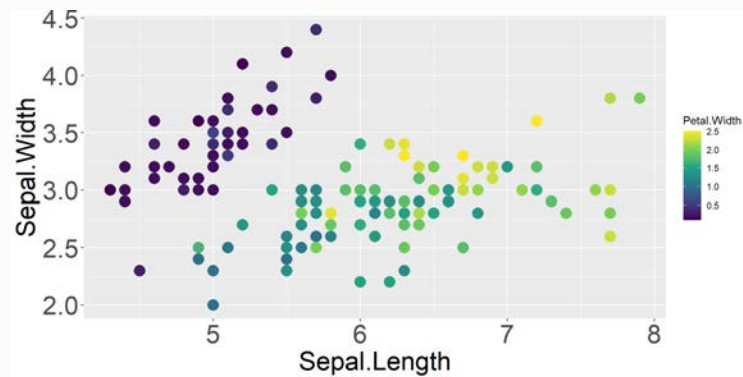
1. La Structure



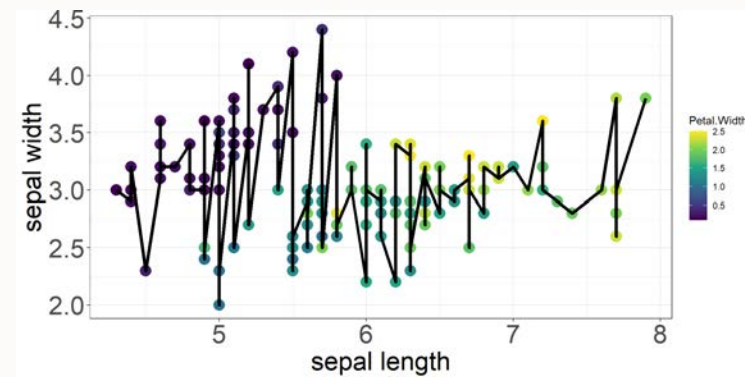
2. Le Type de représentation



3. Les Couleurs



4. Le Lifting final



Ggplot pas à pas : La Structure

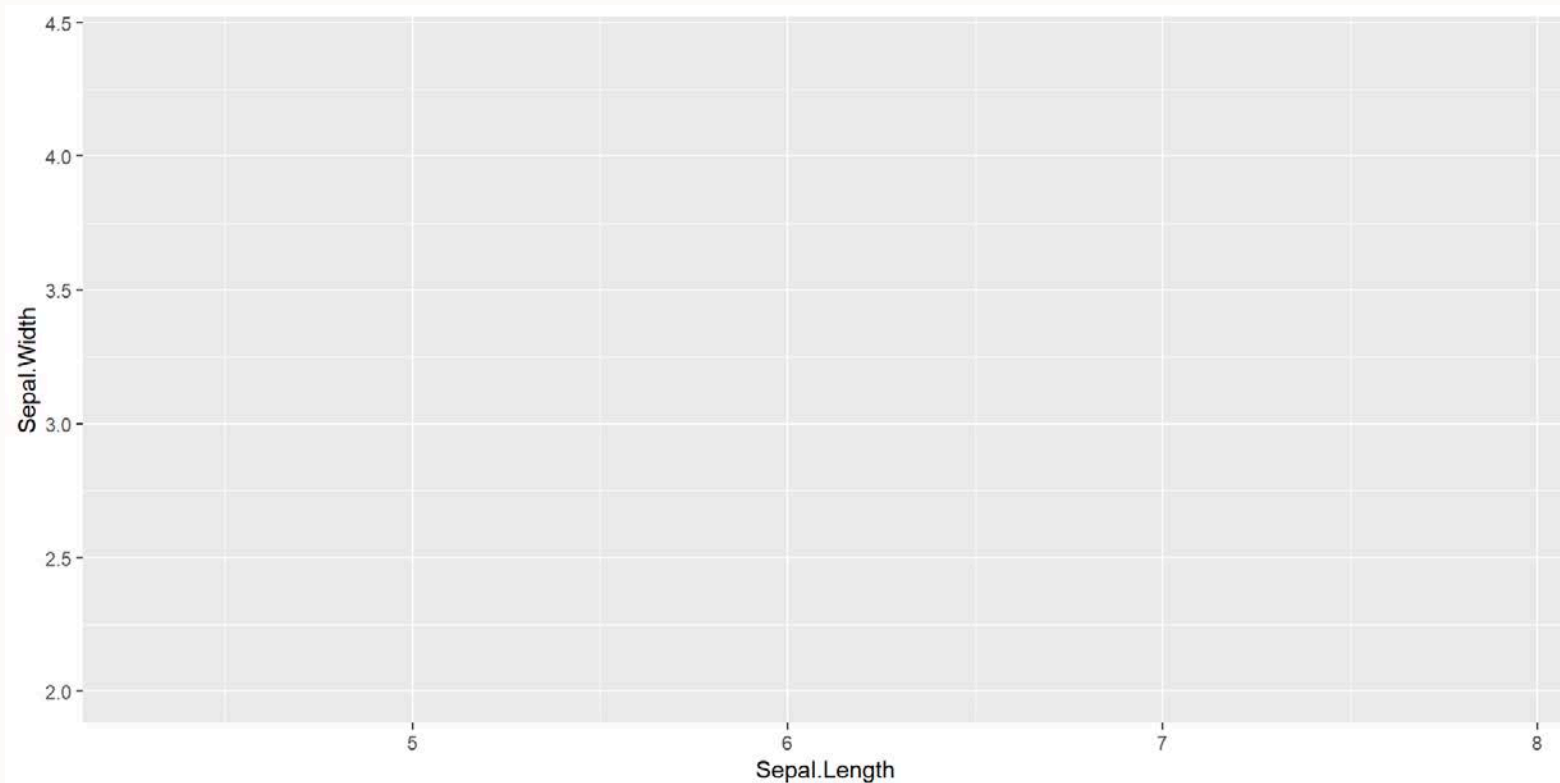
Pour la structure de la figure, la fonction à toujours utiliser est `ggplot()` . Les arguments principaux sont :

- `data = nom.du.dataframe`
- `mapping = aes(x = variable.x, y = variable.y)`
 - Ici le `aes` signifie **aesthetic**
 - On verra que d'autres options sont disponibles dans `aes` comme la taille (`size`), la couleur (`color`), la forme (`shape`)



```
1 data <- iris
2 p1 <- ggplot(data, mapping = aes(x = Sepal.Length, y = Sepal.Width))
3 p1
```

::: {.cell} ::: {.cell-output-display}



...



Ggplot pas à pas : Choix du plot

Plusieurs représentations existent selon le type de variables que l'on a :

1. Une variable continue

- `geom_histogram()`
- `geom_density()`

2. Deux variables continues

- `geom_point()`
- `geom_line()`
- `geom_area()`

3. Une discrète & une continue

- `geom_boxplot()`
- `geom_violin()`
- `geom_col()`

4. Données spatiales

- `geom_polygon()`
- `geom_sf()`



Ggplot pas à pas : Choix du plot

L'exemple ici va mener à la réalisation d'un **nuage de points**. Mais `ggplot2` fonctionne de la même façon pour construire un **diagramme en barres**, une **boite à moustache** ou autres.

- `aes()` dans `ggplot()`

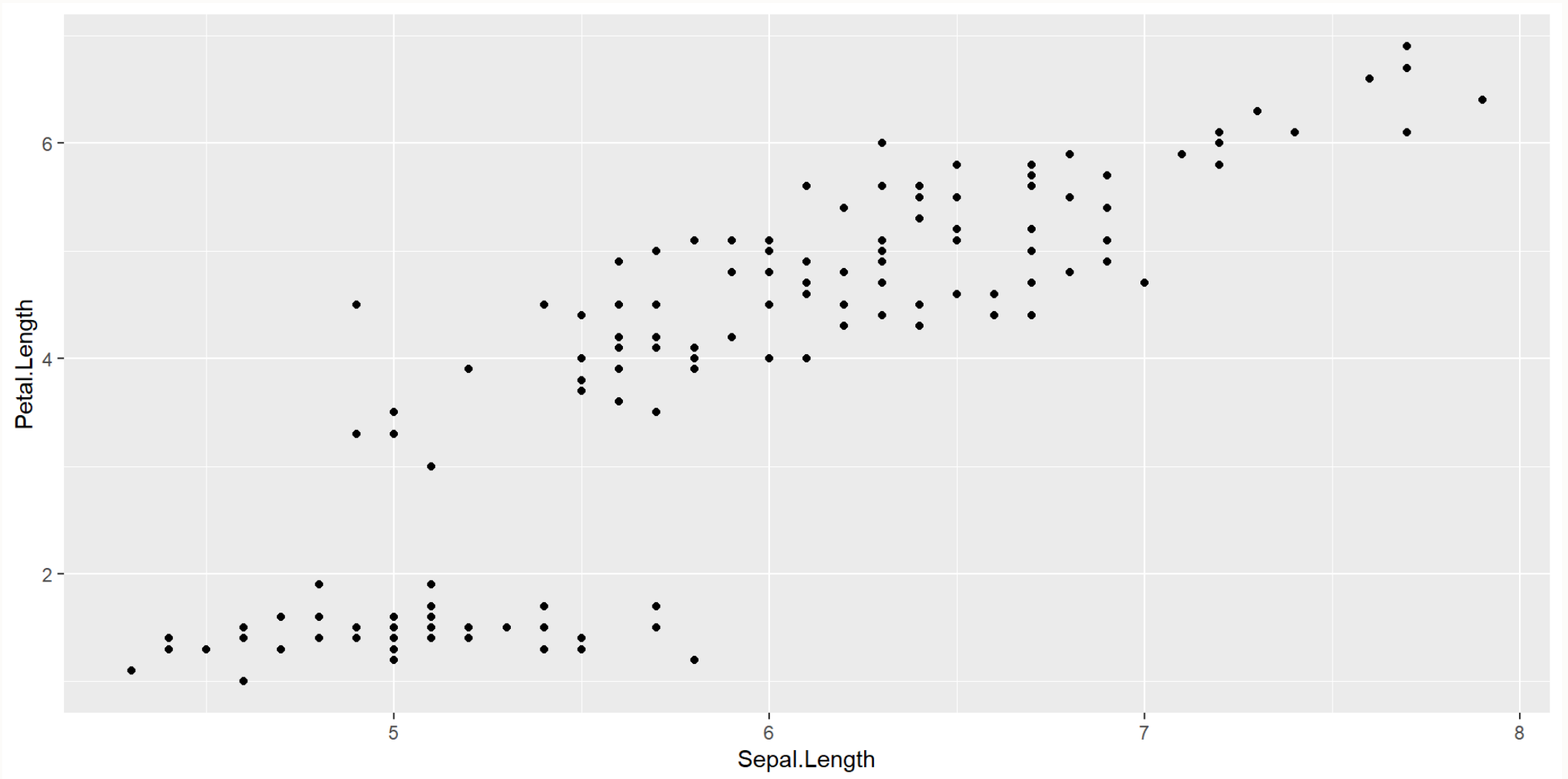
```
1 couche.2 <- ggplot(data = data,  
2                   mapping = aes(x = Sepal.Length,  
3                                y = Petal.Length)) +  
4   geom_point()
```

- `aes()` dans `geom_point()`

```
1 # Autre solution  
2 couche.2.2 <- ggplot() +  
3   geom_point(data = data_crabs,  
4             mapping = aes(x = Sepal.Length,  
5                           y = Petal.Length))
```



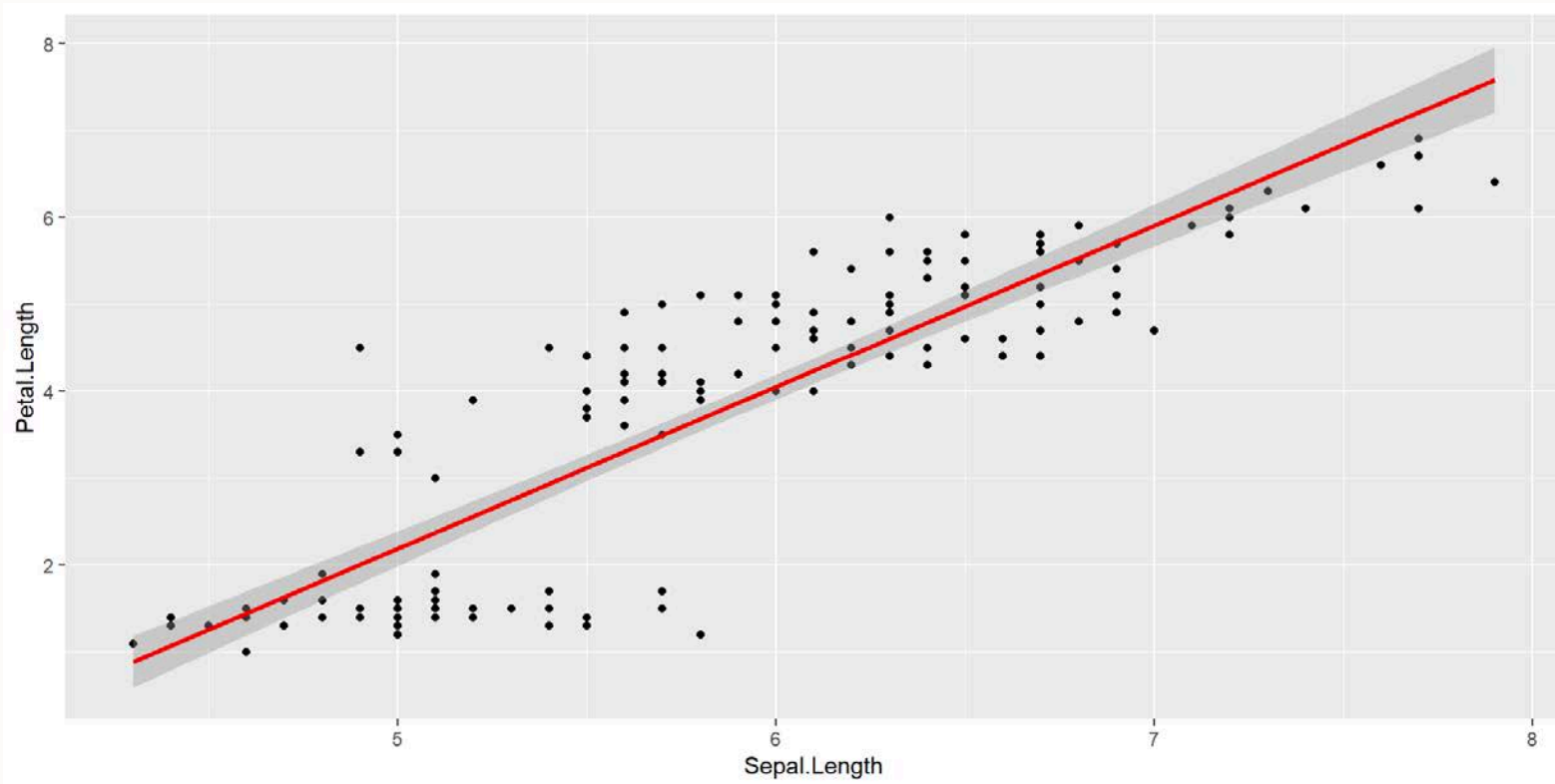
Ggplot pas à pas : Choix du plot



Ggplot pas à pas : Modèle linéaire

Traçons un modèle linéaire

```
1 couche.3 <- couche.2 +  
2   geom_smooth(color = "red", method = "lm") # le modèle linéaire
```

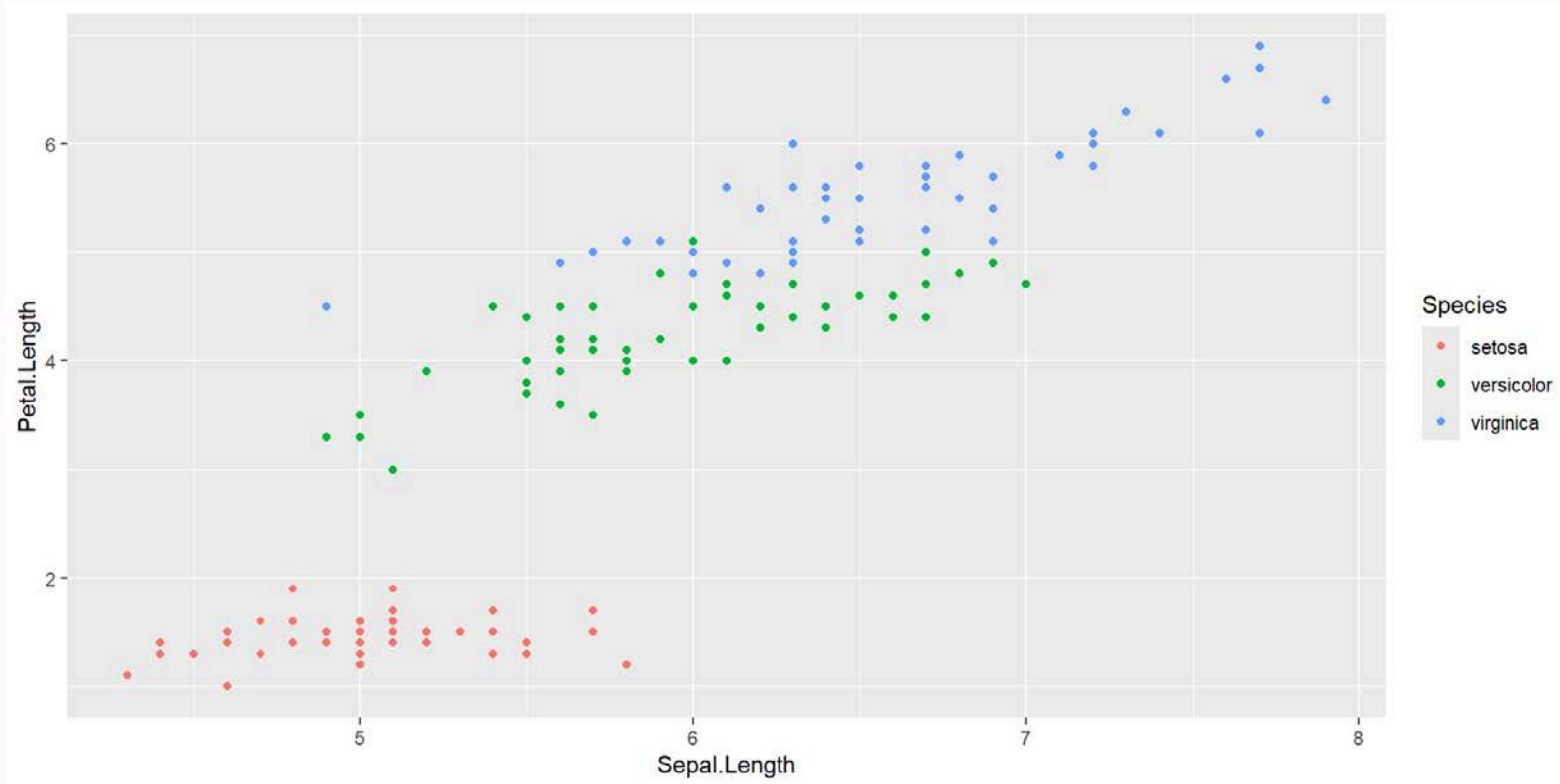


Ggplot pas à pas : La couleur

Donnons une couleur par espèce. Comme c'est lié à l'esthétisme de la figure, il faut retourner à l'argument `color` dans la fonction `aes()`.

```
1 couche.4 <- ggplot() +  
2   geom_point(data = data,  
3             mapping = aes(x = Sepal.Length,  
4                           y = Petal.Length,  
5                           color = Species))
```

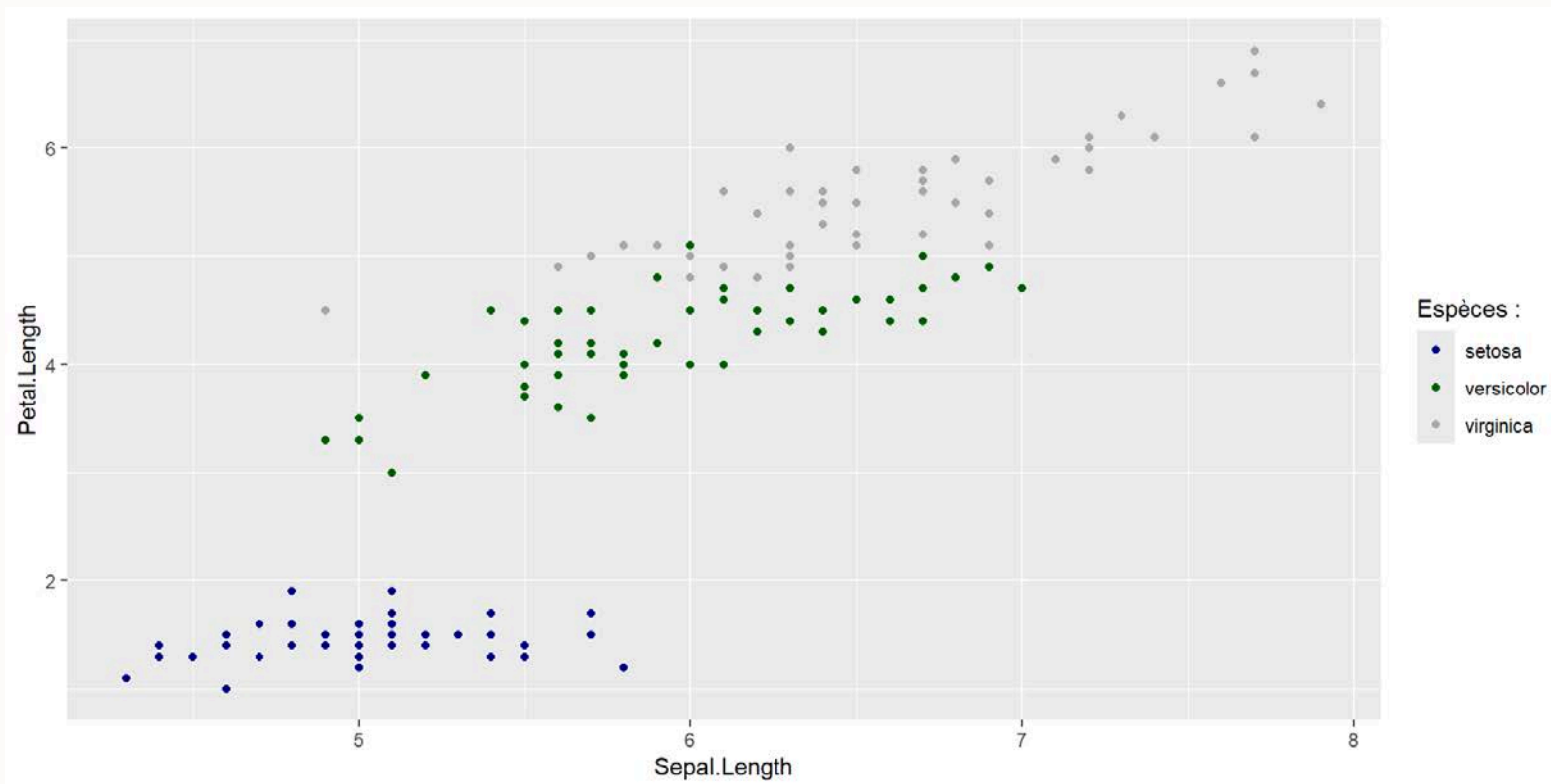




Ggplot pas à pas : Couleurs & légende

Pour cela, on peut utiliser la fonction `scale_color_manual()` et les arguments `name` et `values`.

```
1 couche.5 <- couche.4 +  
2   scale_color_manual(name = "Espèces :",  
3                     values = c("setosa" = "darkblue",  
4                               "versicolor" = "darkgreen",  
5                               "virginica" = "darkgrey"))
```



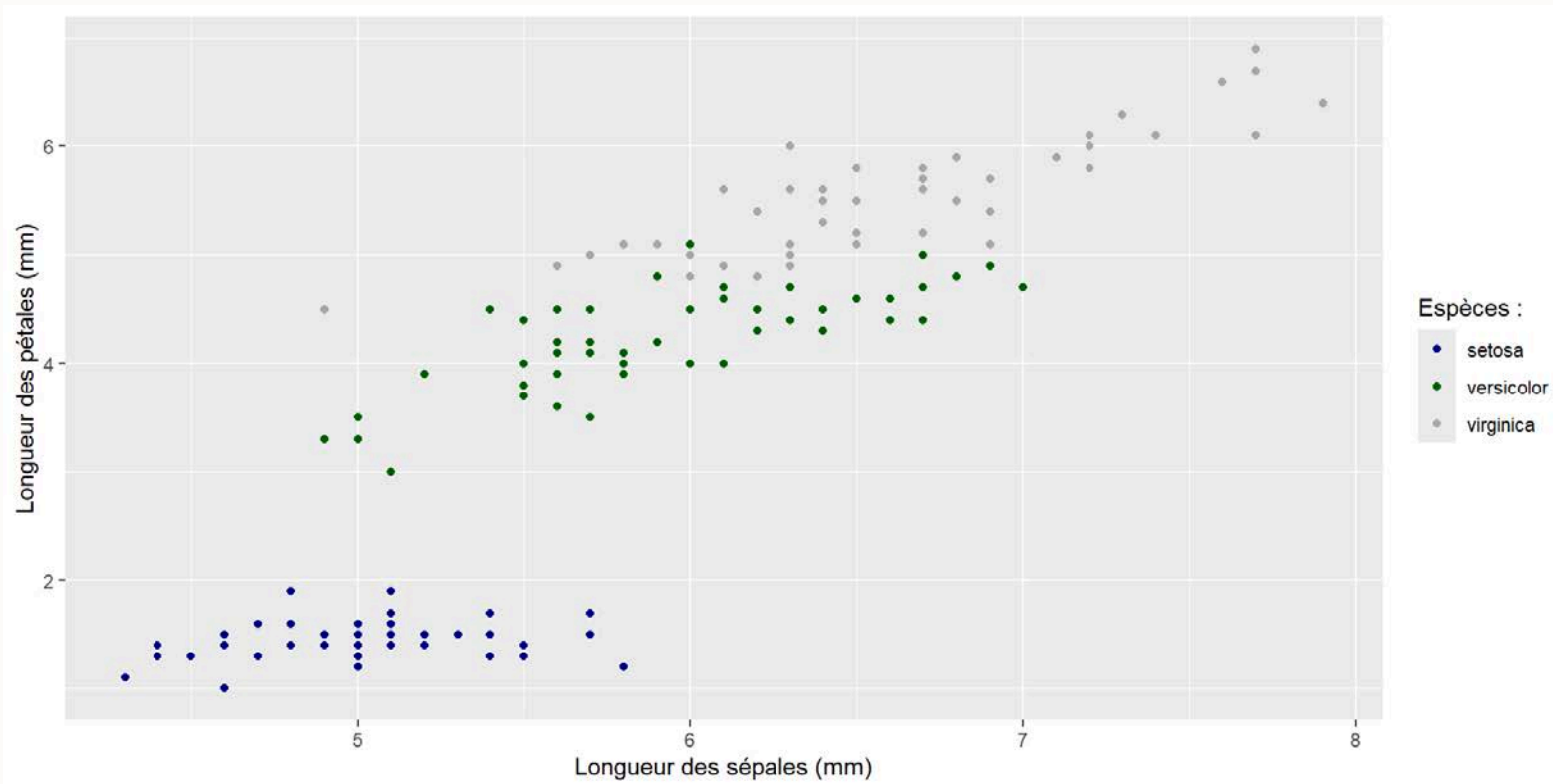
Une autre façon de faire est d'utiliser la fonction `labs` qui permet de changer les diverses légendes (axes x et y + légende de couleur)

<https://github.com/umr-marbec/tidyverse-training>



Ggplot pas à pas : Légende v.2

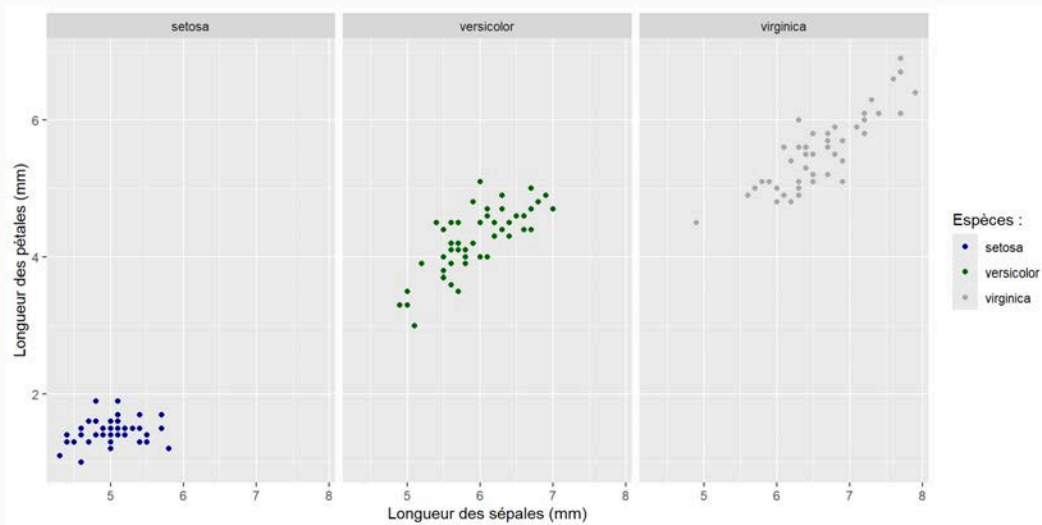
```
1 couche.6 <- couche.5 +  
2   labs(x      = "Longueur des sépales (mm)",  
3        y      = "Longueur des pétales (mm)",  
4        color  = "Espèces :")
```



Ggplot pas à pas : le FACET

La fonction `facet_grid()` permet de générer X plots pour chaque valeur d'une variable qualitative (i.e. l'espèce).

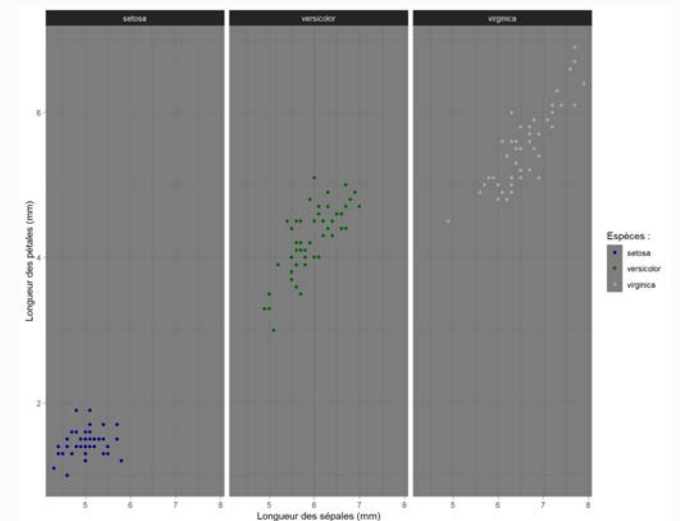
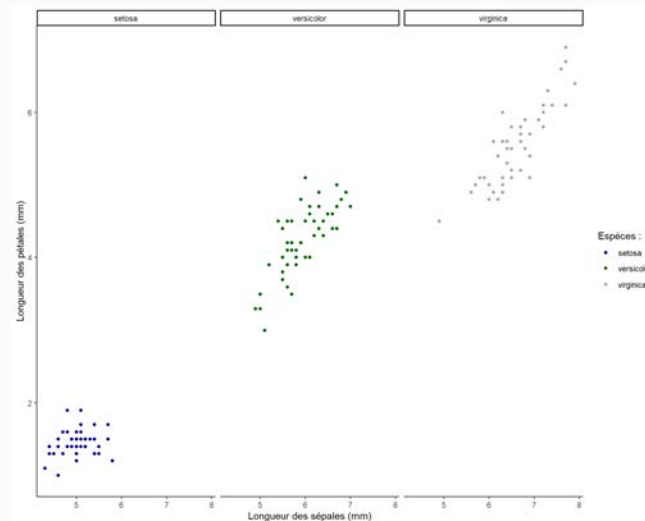
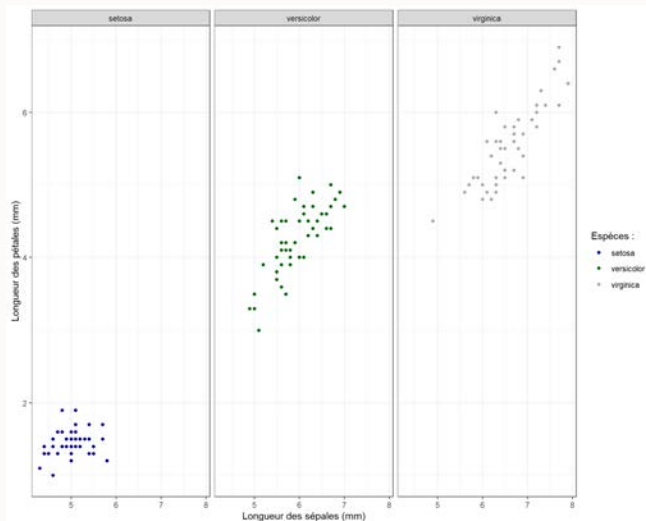
```
1 couche.7 <- couche.6 + facet_grid(~Species)
```



Ggplot pas à pas : l'arrière plan

On utilise une des fonctions `theme_*`.

- 1 `couche.7 + theme_bw()`
- 2 `couche.7 + theme_classic()`
- 3 `couche.7 + theme_dark()`



Un tas de `theme_*` existent pour obtenir différentes formats d'arrière plan (voir [ici](#)). A vous de choisir votre préféré.

<https://github.com/umr-marbec/tidyverse-training>

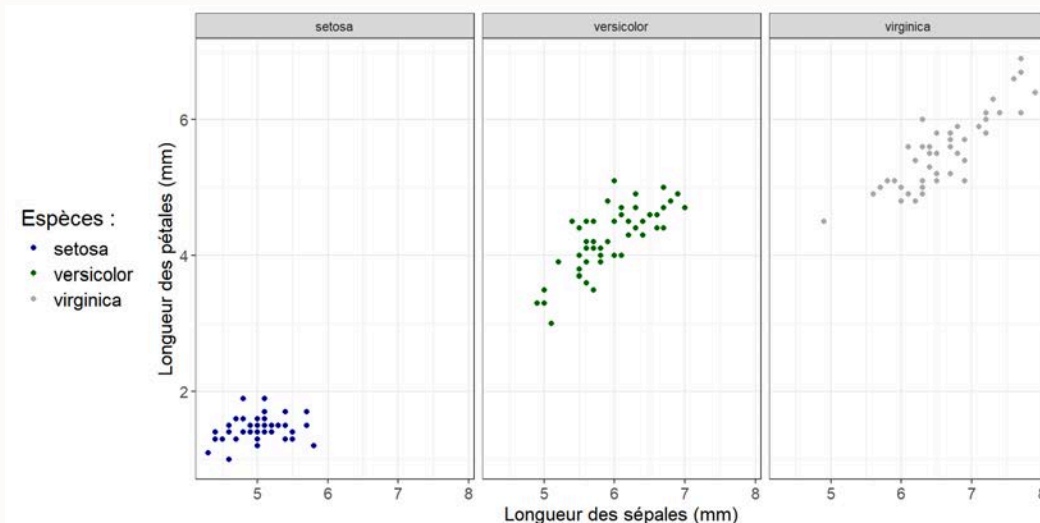


Ggplot pas à pas : le lifting final

La fonction `theme()` permet de lifter BEAUCOUP de choses (`?theme`).

Elle doit toujours apparaître en dernier de façon à ne pas être écrasée par une autre couche.

```
1 couche.8 <- couche.7 +  
2   theme_bw() +  
3   theme(axis.text      = element_text(size = 11),  
4         axis.title     = element_text(size = 13),  
5         legend.position = "left",  
6         legend.text    = element_text(size = 13),  
7         legend.title    = element_text(size = 15))
```



Enregistrer ses figures

Pour enregistrer votre ggplot, vous pouvez utiliser la fonction `ggsave()` :

- `plot` : le nom du ggplot à enregistrer.
- `filename` : le_chemin_daccess/le_nom_denregistrement.EXTENSION
- `width` & `height` : la largeur et hauteur d'enregistrement.

```
1 ggsave(filename = "le_chemin_vers_le_bon_dossier",  
2         plot      = couche.8,  
3         width     = 8,  
4         height    = 5)
```



Exercice 6.1

En utilisant les fonctions vues précédemment - Quel est le pays où la biomasse totale par transect est la plus élevée en **2011** ? - Formatez le jeu de données issues de l'**exercice 3.5** - représentez la structure du plot



Exercice 6.1 : Correction

En utilisant les fonctions vues précédemment - Quel est le pays où la biomasse totale par transect est la plus élevée en **2011** ? - Formatez le jeu de données issues de l'exercice 3.5 - représentez la structure du plot

```
1 biomass_exercice6_1 <- abundance_biomass_survey_exercice3_5 %>%
2   mutate(survey_date_as_date = dmy(survey_date),
3          survey_year = year(survey_date_as_date)) %>%
4   # 1 - Filter les données obtenues en 2011
5   filter(survey_year == 2011) %>%
6   # 2 - Grouper les données pour obtenir la biomasse totale
7   #     par transect dans chacun des pays
8   group_by(country, survey_id) %>%
9   summarise(biomass = sum(biomass))
```

```
1 # Représentation de la structure du plot
2
3 p6_1 <- ggplot(data = biomass_exercice6_1, aes(x = country, y = biomass))
```



Exercice 6.2

- Représentez les données sous forme de boîte à moustache
-



Exercice 6.2

- Représentez les données sous forme de boîte à moustache

```
1 p6_2 <- p6_1 +  
2   geom_boxplot()
```



Exercice 6.3

Ce ggplot n'est cependant pas entièrement satisfaisant ...

Nous avons du mal à savoir visuellement quelle est la 2nd ou la 3ème nation avec le plus de biomasse.

Une solution est d'utiliser la fonction `reorder()` pour ordonner l'axe des x.

- Utilisez cette fonction pour ordonner l'axe des x de façon décroissante selon la valeur **médiane**.

-

N'hésitez pas à regarder l'aide `?reorder` ou à chercher sur internet.

```
1 # Utilisation de reorder
```



Exercice 6.3 : Correction

Ce ggplot n'est cependant pas entièrement satisfaisant ...

Nous avons du mal à savoir visuellement quelle est la 2nd ou la 3ème nation avec le plus de biomasse.

Une solution est d'utiliser la fonction `reorder()` pour ordonner l'axe des x.

- Utilisez cette fonction pour ordonner l'axe des x de façon décroissante selon la valeur **médiane**.

-

N'hésitez pas à regarder l'aide `?reorder` ou à chercher sur internet.

```
1 # Reorder en fonction de la valeur médiane
2 p6_3 <- ggplot(data = biomass_exercice6_1) +
3   geom_boxplot(aes(x = reorder(country, -biomass, FUN = median), y = biomass))
4 p6_3
```



Exercice 6.4

Dans la fonction `element_text()`, un argument permet de tourner les légendes des axes pour que les pays soient plus lisibles. Trouvez cet argument et appliquez une rotation de 45° à la légende de l'axe x. Choisissez votre arrière plan préféré.



Exercice 6.4 : Correction

Dans la fonction `element_text()`, un argument permet de tourner les légendes des axes pour que les pays soient plus lisibles. Trouvez cet argument et appliquez une rotation de 45° à la légende de l'axe x. Choisissez votre arrière plan préféré.

```
1 # Solution
2 p6_4 <- p6_3 +
3   theme_bw() +
4   theme(axis.text.x = element_text(angle = 45, hjust = 1))
5 p6_4
```



Exercice 6.5

Enregistrez la figure **p6_4** au format **.png**, avec une largeur et hauteur appropriées pour lire correctement le plot.



Exercice 6.5 : Correction

Enregistrez la figure **p6_4** au format **.png**, avec une largeur et hauteur appropriées pour lire correctement le plot.

```
1 # Solution
2 ggsave(filename = "figures/boxplot_biomass_vs_country.png",
3         plot      = p6_4,
4         width     = 8,
5         height    = 5)
```



Une petite subtilité concernant les couleurs

fill vs. color

Ggplot fait la différence entre :

- la couleur des contours → `color = ...`
- la couleur de remplissage → `fill = ...`

```
1 couche.9 <- ggplot() +  
2   geom_violin(data = data,  
3               mapping = aes(x = Species,  
4                             y = Petal.Length,  
5                             color = Species)) +  
6   theme_bw()
```

```
1 couche.9_2 <- ggplot() +  
2   geom_violin(data = data,  
3               mapping = aes(x = Species,  
4                             y = Petal.Length,  
5                             fill = Species)) +  
6   theme_bw()
```



Une petite subtilité concernant les couleurs

fill vs. **color**



Une petite subtilité concernant les couleurs

fill vs. **color**



Une petite subtilité concernant les couleurs

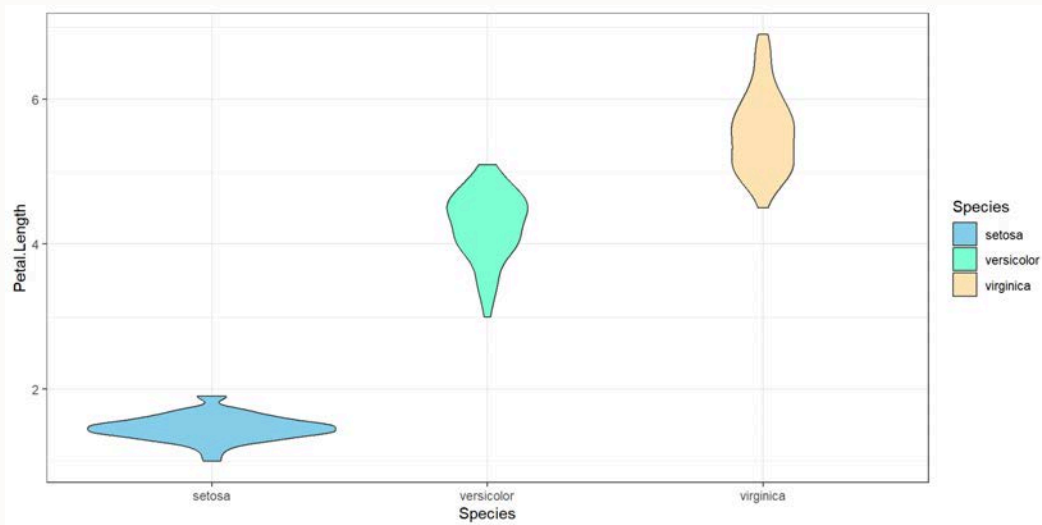
Pour choisir ses couleurs de contour vs. de remplissage, on utilise les fonctions `scale_color_manual()` vs. `scale_fill_manual()`.

```
1 couche.10 <- couche.9_2 +  
2   scale_fill_manual(values = c("setosa" = "skyblue",  
3                               "versicolor" = "aquamarine",  
4                               "virginica" = "moccasin"))
```



Une petite subtilité concernant les couleurs

Pour choisir ses couleurs de contour vs. de remplissage, on utilise les fonctions `scale_color_manual()` vs. `scale_fill_manual()`.



Exercice 6.6

- Reconstituez la figure p6_4 en **remplissant** chaque boîte d'une couleur pour chaque continent.
- Ne garder que les pays suivants : Espagne, Etats-Unis, Italie, Costa Rica et Panama
- Nommez la colonne `continent`



Exercice 6.6 : Correction

- Reconstituez la figure p6_4 en **remplissant** chaque boîte d'une couleur pour chaque continent.
- Ne garder que les pays suivants : Espagne, Etats-Unis, Italie, Costa Rica et Panama
- Nommez la colonne `continent`
- Renommez les axes `Country` et `Biomass (g)`

```
1 # Solution
2 biomass_exercice6_6 <- biomass_exercice6_1 %>%
3   filter(country %in% c("Spain", "United States", "Italy", "Costa Rica", "Panama")) %>%
4   mutate(continent = case_when((country == "Spain" | country == "Italy") ~ "Europe",
5                                 country == "United States" ~ "North America",
6                                 TRUE ~ "South America"))
```



Exercice 6.6 : Correction

- Reconstituez la figure p6_4 en **remplissant** chaque boîte d'une couleur pour chaque continent.
- Ne garder que les pays suivants : Espagne, Etats-Unis, Italie, Costa Rica et Panama
- Nommez la colonne `continent`
- Renommez les axes `Country` et `Biomass (g)`

```
1 # Solution pour le plot
2 p6_6 <- ggplot() +
3   geom_boxplot(data = biomass_exercice6_6,
4                 mapping = aes(x = reorder(country, -biomass, FUN = median),
5                               y = biomass,
6                               fill = continent)) +
7   scale_fill_manual(values = c("Europe" = "skyblue",
8                                "South America" = "aquamarine",
9                                "North America" = "moccasin")) +
10  labs(x = "Country", y = "Biomass (g)") +
11  theme_bw()
```



Une autre subtilité concernant les couleurs

```
1 couche.4_2 <- ggplot() +  
2   geom_point(data = data,  
3             mapping = aes(x = Sepal.Length,  
4                           y = Petal.Length,  
5                           color = Species)) +  
6             scale_color_manual(values = "blue")
```

Error in `palette()`: ! Insufficient values in manual scale. 3 needed but only 1 provided. Run `rlang::last_trace()` to see where the error occurred.

On ne peut pas donner une seule couleur à une variable continue !



Couleurs & variables continues

Il existe 4 fonctions principales qui permettent de construire des palettes de couleurs continues :

- `scale_fill_continuous` : avec une palette par défaut (nuances de bleus)
- `scale_fill_viridis_c` : Colorblind friendly.
- `scale_fill_gradient` : permet de créer ses propres gradients de couleurs. Des variantes existent selon le nombre de couleurs à définir. Il s'agit de `scale_fill_gradient2()` et `scale_fill_gradientn()`.



Couleurs & variables continues

Testons les une par une à partir de la figure suivante

```
1 plot <- ggplot(faithfuld, aes(waiting, eruptions, fill = density)) +  
2   geom_raster() +  
3   scale_x_continuous(NULL, expand = c(0, 0)) +  
4   scale_y_continuous(NULL, expand = c(0, 0)) +  
5   theme(legend.position = "none")
```



Couleurs & variables continues

scale_color_continuous

La plus simple, mais pas beaucoup d'options. C'est aussi la palette de couleur par défaut.

```
1 plot  
2 plot + scale_fill_continuous()
```

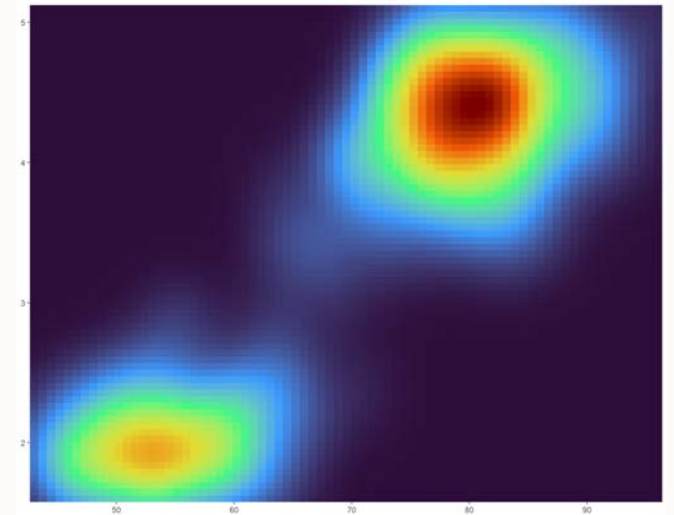
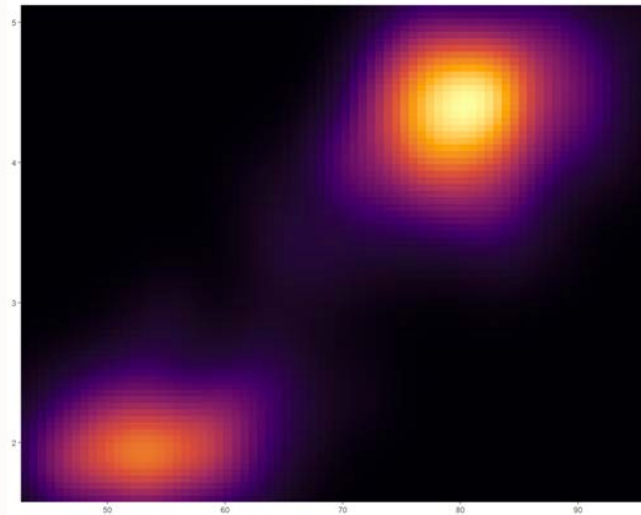
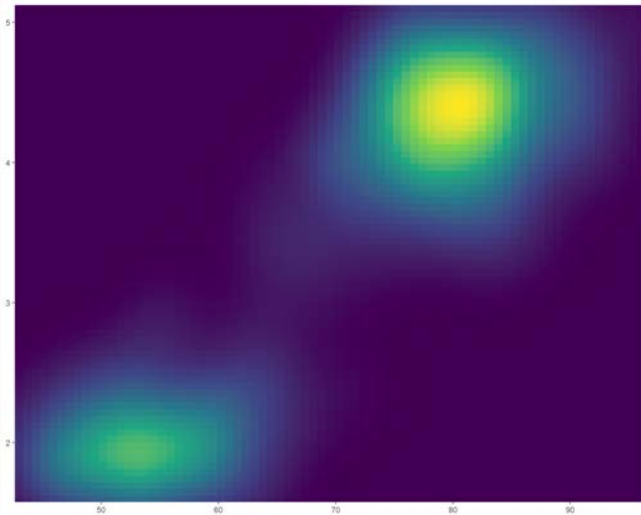


Couleurs & variables continues

scale_color_viridis_c

De belles palettes dont les nuances sont captées par les personnes daltoniennes.

```
1 plot + scale_fill_viridis_c(option = "viridis")  
2 plot + scale_fill_viridis_c(option = "inferno")  
3 plot + scale_fill_viridis_c(option = "turbo")
```



Info générale sur les palettes

Petite info utile, toutes les palettes peuvent passer le test *ColorBlind Friendly* :

```
1 color_pal <- RColorBrewer::brewer.pal(10, name="Spectral")
2 colorBlindness::displayAllColors(color_pal)
```



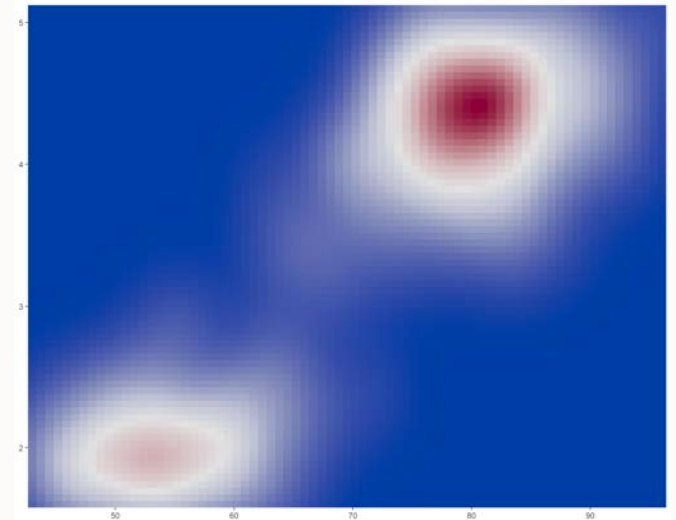
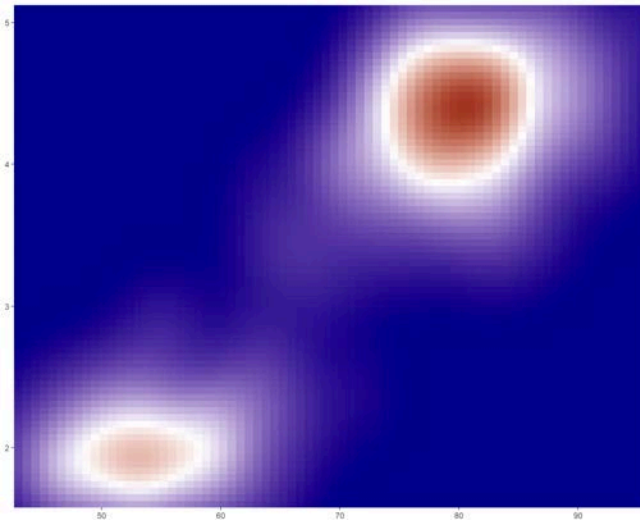
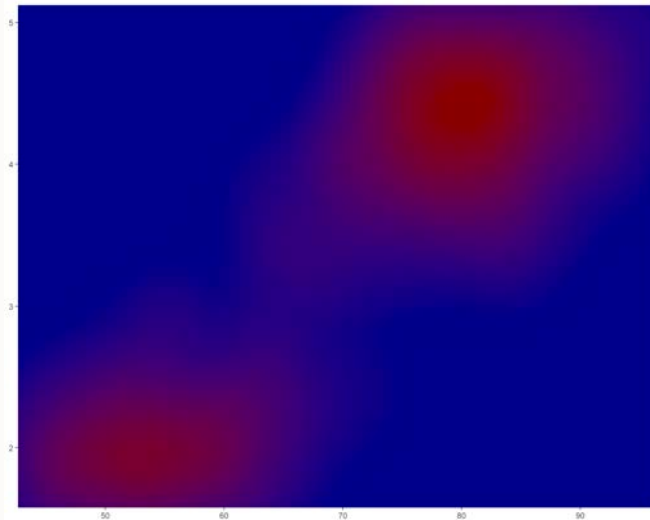
Couleurs & variables continues

scale_fill_gradient()

Permet de créer ses propres palettes de couleurs, à partir de 2, 3 ou n couleurs :

- `scale_fill_gradient()` avec les arguments `low` et `high`.
- `scale_fill_gradient2()` avec les arguments `low`, `mid` et `high`.
- `scale_fill_gradientn()` avec l'argument `colours`.

```
1 plot + scale_fill_gradient(low = "darkblue", high = "darkred")
2 plot + scale_fill_gradient2(low = "darkblue", mid = "white", high = "darkred", midpoint = 0.02)
3 plot + scale_fill_gradientn(colours = colorspace::diverge_hcl(7))
```



Couleurs & variable groupée en classes

Il existe 3 fonctions principales qui permettent de construire ce type de palettes de couleurs :

- `scale_fill_binned` : avec une palette par défaut (nuances de bleus)
- `scale_fill_fermenter` : utilise les `BrewerPalettes` (voir [ici](#) pour plus d'infos).
Pour voir les palettes => `RColorBrewer::display.brewer.all()`
- `scale_fill_steps` : permet de créer sa propre palette avec un nombre de classes précis. Des variantes existent selon le nombre de couleurs à définir. Il s'agit de `scale_fill_steps2()` et `scale_fill_stepsn()`.

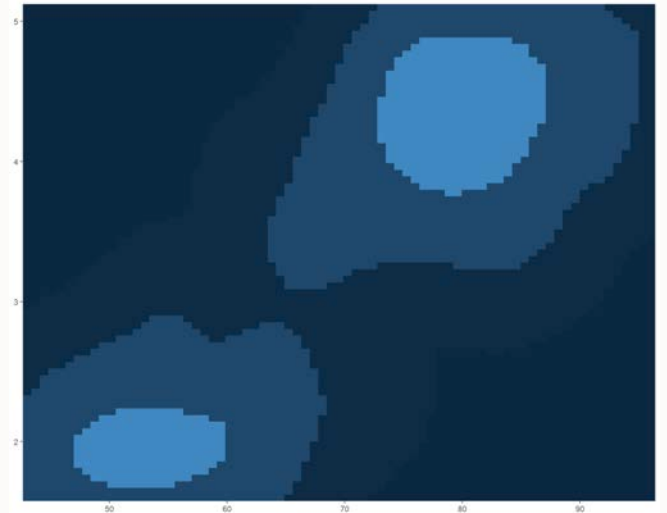
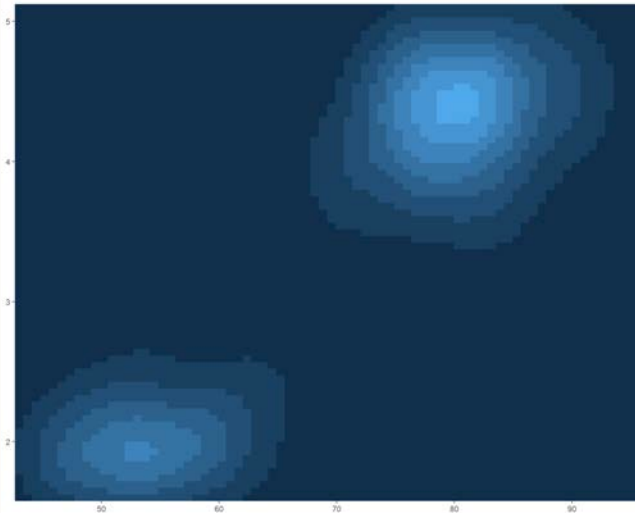
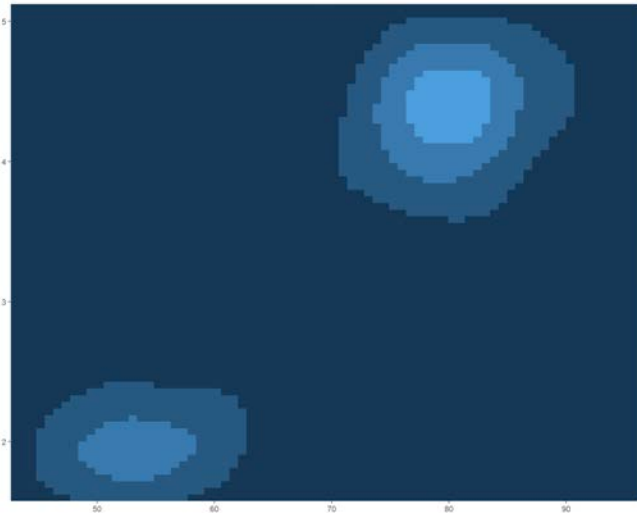


Couleurs & variable groupée en classes

`scale_fill_binned()`

Les couleurs sont par défaut une nuance de bleus. Certaines précisions peuvent cependant être apportées.

```
1 plot + scale_fill_binned(n.breaks = 4)
2 plot + scale_fill_binned(n.breaks = 7)
3 plot + scale_fill_binned(breaks = quantile(faithfuld$density, probs = (seq(0,1,0.3))))
```



Couleurs & variables discrètes

Les principales fonctions utilisées permettant de construire des palettes de couleurs continues :

- `scale_fill_discrete` : Couleurs par défaut
- `scale_fill_manual` : choix de la couleur parmi toutes les couleurs existantes.
- `scale_fill_brewer` : utilise les [BrewerPalettes](#) (voir [ici](#) pour plus d'infos).
Equivalent de `scale_fill_distiller` pour les variables discrètes.



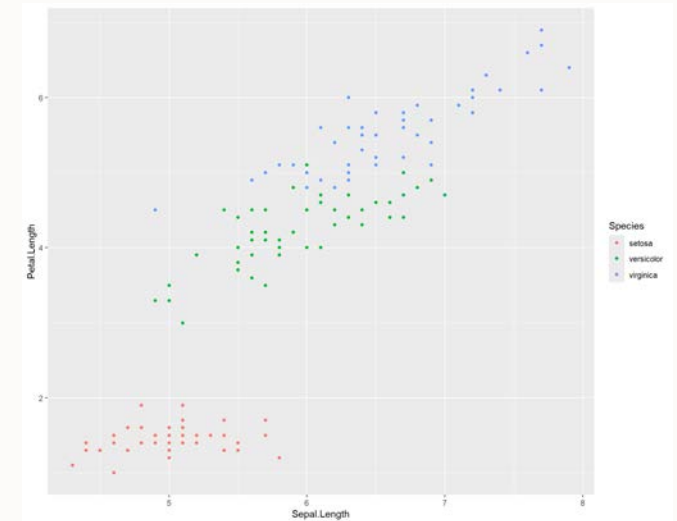
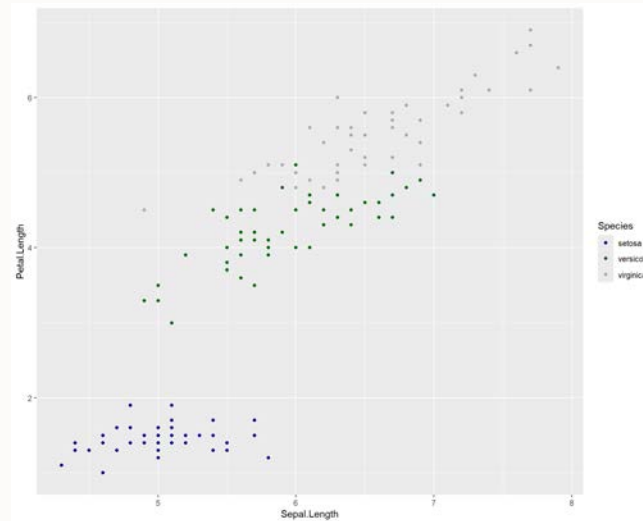
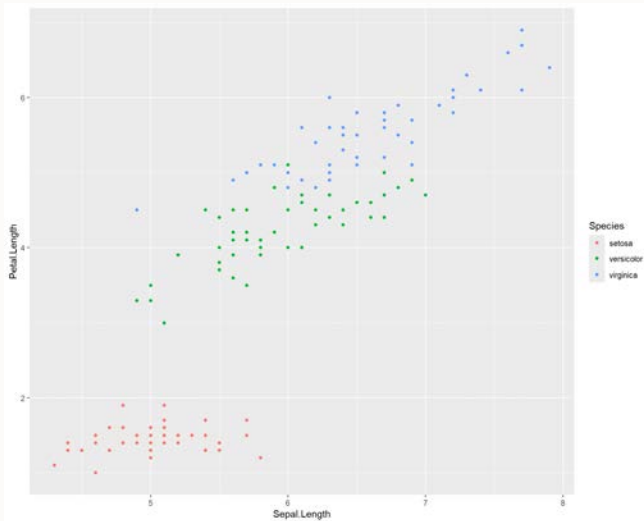
Couleurs & variables discrètes

```
1 plot <- ggplot() +  
2   geom_point(data = data,  
3             mapping = aes(x = Sepal.Length,  
4                           y = Petal.Length,  
5                           color = Species))
```



Couleurs & variables discrètes

```
1 plot + scale_color_discrete()  
2 plot + scale_color_manual(values = c("setosa" = "darkblue", "versicolor" = "darkgreen", "virginica" = "darkgray"))  
3 plot + scale_fill_brewer(palette = "Set2")
```



Ressources utilisées

<https://ggplot2-book.org/scale-colour.html#colour-discrete>

<https://stackoverflow.com/questions/70942728/understanding-color-scales-in-ggplot2>

<https://colorbrewer2.org/#type=sequential&scheme=BuGn&n=3>



Diagrammes en barres & barres d'erreurs

Pour ajouter les barres d'erreurs, on utilise la fonction `geom_errorbar()`.

L'utilisation de cette fonction nécessite le calcul préalable de l'écart-type.

```
1 # Formatage des données
2 data_mean <- data %>%
3   group_by(Species) %>%
4   summarise(sepal_mean = mean(Sepal.Length),
5             sepal_sd   = sd(Sepal.Length))
```

```
1 # Réalisation du diagramme en barres
2 pbar <- ggplot(data_mean) +
3   geom_col(aes(x = Species, y = sepal_mean)) +
4   geom_errorbar(aes(x = Species, ymin = sepal_mean-sepal_sd, ymax = sepal_mean+sepal_sd)) +
5   theme_bw()
```



Diagrammes en barres & barres d'erreurs



Exercice 6.7

A partir des données **biomass_exercice6_6**, construire un diagramme en barres de la biomasse moyenne par transect. Classer l'axe des x dans l'ordre **croissant**. Trouvez la façon de rendre les barres moins larges et de changer leur couleur.



Exercice 6.7 : Correction

A partir des données de l'exercice 6_6, construire un diagramme en barres de la biomasse moyenne par transect. Classer l'axe des x dans l'ordre **croissant**. Trouvez la façon de rendre les barres moins larges et de changer leur couleur.

```
1 biomass_exercic6_7 <- biomass_exercice6_6 %>%
2   # 1 - Grouper les données par pays
3   group_by(country) %>%
4   # 2 - Prendre la valeur de biomasse moyenne et l'écart-type de l'ensemble
5   #   des transects propres à chaque pays
6   summarise(biom_mean = mean(biomass),
7             biom_sd   = sd(biomass))
```



Exercice 6.7 : Correction

A partir des données de l'exercice 6_6, construire un diagramme en barres de la biomasse moyenne par transect. Classer l'axe des x dans l'ordre **croissant**. Trouvez la façon de rendre les barres moins larges et de changer leur couleur.

```
1 # Figure
2 p6_7 <- ggplot(biomass_exercic6_7) +
3   geom_col(aes(x = reorder(country, biom_mean), y = biom_mean)) +
4   geom_errorbar(aes(x = country, ymin = biom_mean-biom_sd, ymax = biom_mean+biom_sd),
5                 width = 0.2, colour = "red") +
6   labs(x = "Country", y = "Biomass (g)") +
7   theme_bw()
8 p6_7
```

